

Cover Letter

We thank all reviewers for their constructive suggestions. Below, we summarize our responses to the revision requests from all reviewers. Changes to the paper are marked in **blue** for Reviewer 1, **violet** for Reviewer 2, and **teal** for Reviewer 3. A change requested by several reviewers is marked once, in the color of the reviewer under whom it is described first in this letter, and cross-referenced for the other reviewers. Every change additionally carries a margin tag naming the comment it answers (e.g., **R2.O1**), printed next to the changed text. The revised paper follows this letter and restarts its own page numbering at page 1.

Meta-Reviewer:

C1. Carefully address R1.O1 with all the required details, and address R3.O5.

Response: R1.O1 is addressed with a full technical explanation in Section 5.2; please see our response to R1.O1 below. For R3.O5, we agree that conditions (7)–(10) certify the join skeleton but not, by themselves, the output multiset. The output-certification step for plain joins is now fully specified in the new Section 6.4, in the same style as the other gates: private witness (occurrence vectors), verification conditions (11)–(15), realization by the existing primitives, and correctness and complexity analyses; please see our response to R3.O5 below. Its implementation accompanies the updated artifact.

Reviewer 1:

O1 (and Q1). It is unclear how Eq. (2) can certify a permutation or a subset predicate: the index i differs under a permutation, and storing a random permutation already requires $O(n \log n)$ information, so the claimed $O(n)$ cost and the secrecy of the permutation need justification.

Response: We clarify that under ZK-SNARKs the circuit need not compute the output: the prover computes everything off-circuit and supplies it as a private witness, and the circuit only verifies that the input and the claimed output are consistent. This is unlike MPC, where no single party sees all the data, the function must be computed obliviously in-circuit, and applying a secret permutation indeed requires an $O(n \log n)$ routing network. Eq. (2) is not a pointwise identity: each side is a grand product over all rows and is therefore invariant to reordering, so it asserts only multiset equality—a statement that is independent of the secret permutation σ . Two size- n multisets are equal if and only if $\prod_i (X + P_i) = \prod_i (X + P'_i)$ as polynomials, which is checked at a random point in $O(n)$ by the Schwartz–Zippel lemma; this is the standard shuffle argument of Neff [35] and Bayer and Groth [8], which also underlies PLONK [19], Plookup [18], and Halo2’s lookup argument [49], and is used by vSQL [52], ZKSQL [32], and PoneglyphDB [26]. Since σ never enters the witness and all columns are hidden by zero knowledge, no proof—nor any sequence of proofs—leaks σ or any intermediate cardinality. The subset predicate adds one constraint per row: with the witness column arranged so that equal values are adjacent and the table column aligned to it, the constraints $P'_0 = Q'_0$ and $(P'_i - Q'_i)(P'_i - P'_{i-1}) = 0$ force, by induction,

every value of P' to appear in Q' . We have incorporated this discussion into Section 5.2 as three new paragraphs, “Why Eq. (2) certifies a secret permutation in $O(n)$,” “Why the permutation stays secret at no extra cost,” and “From permutation to subset,” and have added the two supporting references [8, 35] to the bibliography.

Reviewer 2:

O1. Using DP for cyclic queries contradicts the threat model, since it weakens the zero-knowledge guarantee; moreover, what would the speedup be against a PoneglyphDB that also released DP intermediate results?

Response: We clarify two points. (1) The DP-guided padding is optional. To obtain full obliviousness for cyclic queries, VPJoin uses tree decomposition with worst-case intra-cluster padding, and all cyclic VPJoin bars in Figure 3 use this fully oblivious mode; the DP mode only trades a quantified (ϵ, δ) leakage of the intra-cluster capacities for additional speedup. To remove the impression that DP must be used for cyclic queries, we renamed the gate from “DP-Guided Join Gate (DPJ)” to “Tree-Decomposed Join Gate (TDJ),” retitled Section 7.1, and now present the fully oblivious construction first, with DP-guided padding as an optional mode. (2) A hypothetical PoneglyphDB + DP would remain slower than VPJoin + DP: PoneglyphDB decomposes the k -way join into $k-1$ binary joins and must add noise after each of them, its sensitivity grows with the full-join depth, and its privacy budget must be split across all $k-1$ joins. More noise means more dummy rows, a larger circuit, and a longer proving time. VPJoin, in contrast, adds noise only for the small intra-cluster joins and splits its budget only a few ways. The new Appendix G.3 details this comparison. The rename and the fully-oblivious-by-default framing are propagated through the abstract, Section 1.1, the title, introduction, body and Definition 3 of Section 7, the experimental setup in Section 8.1, and Appendices G and H.2.

O2 (and Q1). Why are PoneglyphDB’s intermediate results padded to the worst case, given that foreign-key joins bound the intermediate sizes by $O(IN)$?

Response: We consider the general setting in which the verifier has no prior knowledge of schema-level constraints such as primary-key/foreign-key (PK/FK) relationships. Assuming that PK/FK constraints are known to the verifier is a simpler setting, since they reveal structural properties of the joins and may partially expose join selectivities; not relying on them is standard in privacy-preserving query answering [29, 45]. Without PK/FK bounds, worst-case intermediate sizes are not $O(IN)$, which is why both systems are evaluated with public size bounds only. This assumption is now stated explicitly in the introduction (Section 1) and in the experiments (Section 8.1); the same change also addresses R3.O4 and R3.Q2.

O3. The coloring of Figure 5 appears to be incorrect.

Response: We thank the reviewer for catching this mistake. The legend labels of the fully oblivious VPJoin bars and of the Revealing Join Size bars were indeed swapped in the plotting script. We fixed the script and regenerated the figure: the tallest bars

are now correctly labeled “VPJoin” (fully oblivious, worst-case padding) and the shortest “Revealing Join Size,” which matches the text. A note in the caption of Figure 5 records the correction, and we adjusted the overhead wording in Section 8.4 so that it matches the plotted numbers exactly.

Reviewer 3:

O1, O3 (and Q1). Explain how the infeasible PoneglyphDB times were obtained, and how the headline “30 years / 10^{13} years” numbers can be validated.

Response: Since the fully padded PoneglyphDB-style circuits cannot be run to completion even at the 60K scale, the PoneglyphDB times in Figure 3 are extrapolated from circuit size and anchored on measured executions: the released PoneglyphDB artifact for the TPC-H queries, and, for the graph queries, which that artifact does not cover, a dedicated PoneglyphDB-style binary-join-chain baseline circuit that we implement in our artifact (`src/graph_sql/pone_baseline.rs`). Concretely, we derive the gate-row count N of the fully padded PLONKish circuit, take the padded power-of-two domain size $G_C = 2^{\lceil \log_2 N \rceil}$, anchor on a measured runnable PoneglyphDB execution on the same server with true unpadded intermediates (yielding $N_0, G_{C,0}, T_0$), and estimate $T_{\text{est}} = T_0 \cdot G_C / G_{C,0}$, exploiting the facts that the MSM cost dominates and that the proving time is near-linear in the domain size. The estimate is conservative in PoneglyphDB’s favor, since the gate structure is identical and neither the FFT growth nor the large-degree overheads are charged to it. The revised paper also makes two points about the anchors explicit. First, the anchor (unpadded) executions are faster than VPJoin because they materialize only the true, small intermediates while revealing every intermediate cardinality, so Figure 3 compares the two systems at the same (fully oblivious) privacy level. Second, since the released PoneglyphDB artifact provides only TPC-H circuits, the graph-query baselines are built from the same binary-join gate library that PoneglyphDB uses, follow its binary-join execution strategy, and are extrapolated identically. We have written the methodology up as a new paragraph, “Estimation Methodology for PoneglyphDB,” in Section 8.1. Finally, every extrapolated number is now labeled as an estimate and points the reader to that paragraph for the assumptions: the motivating example in Section 1, the captions of Figures 3 and 5, and the reported times in Sections 8.2 and 8.4, including the fully oblivious VPJoin times for GQ3 and GQ4.

O2 (and Q3). The released PoneglyphDB implementation scales only the line item table and binds no database commitment; these simplifications must be disclosed.

Response: We confirm this. For consistency with the PoneglyphDB codebase, our experiments inherit its setup: only line item is scaled, with the dimension tables held fixed, and the public input is trivial, with no externally published Commit(D). The revised paper discloses both simplifications where the datasets and the scaling experiment are described (the Datasets paragraph of Section 8.1, a new paragraph “Inherited Setup from the PoneglyphDB Codebase” in the same section, Section 8.3, and the “Artifact status” paragraph of Appendix A), and adds a discussion of how the

headline numbers would change without them. Scaling the dimension tables proportionally grows VPJoin’s cost near-linearly in $IN + OUT$, while PoneglyphDB’s padded intermediates grow multiplicatively, so the gap would widen; binding Commit(D) adds a one-time $O(N)$ setup commitment plus per-proof opening costs that affect both systems identically.

O4 (and Q2). Why does disabling the PK/FK optimizations make sense, and was PoneglyphDB’s circuit modified?

Response: Please see our response to R2.O2 above: we target the general setting in which the verifier has not trustworthiness constraints, which is also the more private setting; this is now stated in the introduction and in the experiments (violet). Regarding Q2, we did not modify PoneglyphDB’s circuit. Following our response to O1 above, the PoneglyphDB numbers in Figure 3 are extrapolated, and the extrapolation sets the padded intermediate capacities from public size bounds, i.e., without PK/FK knowledge, as now described in the estimation-methodology paragraph.

O5. The final join enumeration is under-specified for plain joins: conditions (7)–(10) do not certify the output multiset.

Response: We agree. For plain joins, we add a final output-certification step over the certified clean relations and the reported output: the prover supplies, for each output row, the clean-tuple occurrence vectors that generate it; the circuit copies those clean tuples, binding each of them to its clean relation by a lookup, checks that their shared join keys agree, and checks that they reproduce the claimed output row. Presenting the occurrence vectors in strictly increasing canonical order rules out duplicated rows, and binding the number of output rows to the join cardinality $\sum_{t \in R_{\text{root}}} \mu(t)$, which is certified by the multiplicity propagation of Section 6.3, rules out omitted rows. Together, these determine the exact output multiset at $O(IN + OUT)$ total cost. Section 6.4 gives the full specification: the occurrence vectors carry the clean-tuple indices, which preserves multiset semantics for duplicate tuples; conditions (11)–(15) formalize binding, join-key agreement, row reproduction, canonical ordering, and cardinality binding; and each condition is realized by a primitive already used in Section 6, with its constraint cost stated. In addition, we removed the previously unqualified claim that the join “can be enumerated directly from the reduced relations without further validation”: in the Two-Pass and One-Pass OBJ realizations, in both worked examples, in the roadmap of Section 6, and in the discussion of projection and cross-relation predicates in Appendix D, the enumeration is now described as a prover-side step whose reported output is certified by Section 6.4. Its circuit implementation accompanies the updated artifact.

O6. The title, abstract, and introduction should acknowledge that the cyclic case in DP mode is a relaxation of zero knowledge, and state which queries obtain which guarantee.

Response: We agree that readers should not infer that all reported configurations provide the same guarantee. As detailed in our response to R2.O1, cyclic joins are handled by tree decomposition and remain fully oblivious by default; DP-guided padding is an optional relaxation, used only where it is explicitly indicated (VPJoin + DP). The abstract and the introduction now state this

explicitly, and the experiments state which configuration each reported number uses.

A1. The artifact does not implement the commitment layer of Appendix A; the proofs use the trivial public input.

Response: We confirm this, and it is now disclosed in the paper: the originally released artifact benchmarks the query-verification circuits, with Halo2’s internal IPA commitments providing within-proof binding only. The updated artifact implements the commitment layer of Appendix A as a standalone, additive module (`src/column_commit.rs`, with the harness `src/bin/commit_cost_bench.rs`), leaving every query circuit unchanged so that its cost is measured separately and added on top of any query’s cost. Each attribute column is committed separately, over the same evaluation domain that the query circuits already use ($k = 16$), with a fresh random blinder per column, so the layer reuses the circuits’ existing parameters with no separate large setup, and the published commitments are hiding. The in-circuit consistency step that the reviewer asks for is then realized as a random-point check, applied per query to exactly the columns that the query’s circuit witnesses (Q3: 10, Q5: 16, Q8: 19, Q9: 17, Q18: 8; the two Edge columns for GQ1–GQ4), in that circuit’s own degree: a Fiat–Shamir challenge x is derived from the published commitments, the layout, and the query proof; the prover opens each committed column at x ; and the circuit evaluates its own witness column at x by a Horner accumulation and exposes the value as a public output, so that matching values certify that the query circuit’s witness column equals the committed data. Measured per query at the 60K scale, the setup costs at most 0.07 s and publishes 608 bytes; per proof, the layer adds 1.0–2.2 s of in-circuit prover time over the identical circuit without binding, plus 2.0–4.7 s of openings and at most 0.25 s of verification, for totals of 3.0–6.8 s across Q3–Q18 and 1.5–2.9 s across GQ1–GQ4 on all three datasets, i.e., under 4% of the corresponding measured VPJoin proving times. The harness (`src/bin/query_commit_bench.rs`) emits a CSV with the per-query baseline-versus-bound deltas, so the additional cost is directly comparable against the pure query circuit. Appendix A records all of this in a new “Artifact status” paragraph, and the new module ships with the updated artifact.

A2. No Laplace sampling or sensitivity computation appears in the code; the DP variants use hard-coded padding constants.

Response: The DP mechanism is a prover-side step that is not verified in-circuit: the circuit consumes only the final padded capacity and never samples noise itself. The capacities are computed offline: for each intra-cluster join we calibrate the noise using the truncation-based join mechanism of Donget al. [14], applied as a one-sided (non-negative) upper bound, so that the released $\tilde{m} \geq m$ never underestimates the true cardinality. The originally released artifact hard-coded the resulting capacities, which is why they appeared there as fixed constants. The updated artifact includes the generation code (`dp/noise_generator.py`): it first releases one-sided noisy truncation thresholds for the per-relation maximum join-key frequencies, each of which has global sensitivity 1, and then releases the capacity with one-sided noise calibrated to the join-size sensitivity $\max(\tilde{F}_A, \tilde{F}_B)$; as a single-tuple changes to A moves the join size by at most F_B and as a single-tuple changes to B by at most F_A , so the sensitivity is $\max(F_A, F_B)$ with the released bounds in place

of the private frequencies, and this holds for the self-join graph clusters as well under the per-instance neighboring model. The total privacy cost follows by basic composition, and every released value is a single draw of the mechanism. Appendix G.2 documents the mechanism, and the generation code ships with the updated artifact (`dp/noise_generator.py`, plus a Rust twin `src/dp_noise.rs` that is callable at witness-generation time, so that the DP-mode circuits consume the released capacities directly).

VPJoin: Verifiable and Privacy-Preserving Multi-Way Join via Zero-Knowledge Proofs

Anonymous Author(s)

Abstract

Modern applications increasingly demand both data confidentiality and query integrity in SQL query answering. Zero-knowledge (ZK) proofs address this by allowing a prover to demonstrate the correct execution of SQL queries on private data without revealing the data itself. However, efficiently proving multi-way joins poses a fundamental challenge rooted in a seemingly irreconcilable tension: existing solutions need to materialize all intermediate results and use them to perform the verification. Since intermediate join results are data-dependent and thus privacy-sensitive, such solutions must further pad every intermediate to the worst-case size. As a result, for the join query involving k relations, their verification requires executing a circuit with the size of $O(IN^k)$, where IN is the total input size, rendering even modest multi-way joins (e.g., $k \geq 3$) computationally infeasible.

To address this issue, we present VPJoin, which resolves this tension through a paradigm shift from proving-by-re-execution to proving-by-correctness-verification: rather than materializing all intermediate join results, the prover supplies a more compact witness, while the verification is done through lightweight structural checks. For acyclic joins, we introduce the Oblivious Join Gate (OBJ), which verifies semi-join-based reductions along a join tree and achieves $O(IN + OUT)$ complexity, where OUT is the join result size. OBJ also supports join-aggregate queries with the same cost, where OUT denotes the final aggregated output, by propagating tuple multiplicities without materializing the full join. For cyclic joins, we develop the **Tree-Decomposed Join Gate (TDJ)**, which confines materialization to small clusters via tree decomposition while remaining fully oblivious; an optional DP-guided padding mode further reduces the intra-cluster padding cost in exchange for a quantified (ϵ, δ) leakage of intra-cluster capacities. Experiments on TPC-Hand graph-pattern benchmarks show that VPJoin outperforms the state-of-the-art by 6–18 orders of magnitude in proving time.

Keywords

zero-knowledge proofs, verifiable databases, multi-way joins

ACM Reference Format:

Anonymous Author(s). 2018. VPJoin: Verifiable and Privacy-Preserving Multi-Way Join via Zero-Knowledge Proofs. In Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX). ACM, New York, NY, USA, 28 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Organizations increasingly publish analytical results derived from sensitive databases, including regulatory filings, public-health statistics, and cross-organization analytics. A fundamental question arises: how can a data holder convince external parties that a reported query result is correct, computed faithfully over the committed dataset, without revealing the underlying private records? Valid solution to this problem requires two properties: verifiability (external parties can confirm the result was computed correctly) and privacy (the underlying data records remain hidden during the verification). For example, a government agency may publish unemployment statistics computed from a private census database: external auditors need assurance that the reported figures are accurate, yet must not gain access to any individual record.

To achieve these goals, zero-knowledge (ZK) proofs [20, 22, 24, 30, 33, 37, 42] were introduced. In this setting, there are two parties: a prover, who holds a private database D and claims a query Q together with its result $Q(D)$, and a verifier, who seeks to be convinced that the claim is correct without learning the underlying data. At a high level, the prover first publishes a cryptographic commitment to D , which binds all subsequent proofs to the same database instance. To generate a proof, the prover constructs a private witness consisting of the input database and auxiliary values needed to certify the computation. The relationships between this witness and the claimed result are then encoded as an arithmetic circuit. Using the circuit and the witness, the prover generates a proof π , and the verifier checks π to confirm that $Q(D)$ is indeed the correct output of Q on the committed database D , thereby ensuring verifiability. At the same time, the cryptographic guarantees of zero knowledge ensure that π reveals nothing about D beyond the query result $Q(D)$ itself, thereby preserving privacy. To maintain this guarantee, neither the circuit structure nor the witness size, which directly affects the circuit, should leak private information about the data. In terms of efficiency, verification can often be performed in logarithmic time, while the main bottleneck lies in proof generation, whose cost is roughly linear in the circuit size (see Section 3.2 for more details). Henceforth, we use circuit complexity to refer to the cost of proof generation, since the two are deterministically related and nearly linear.

Applying ZK proofs to query answering over relational databases has been widely studied [26, 32, 52, 53]. Recent ZK database systems such as ZKSQL [32] and PoneglyphDB [26] have demonstrated that ZK proofs for single-table operations such as sort, range check, and binary joins are efficient and practical. However,

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

R2.O1
R3.O6

these works do not efficiently support multi-way joins, despite the fact that multi-way joins are fundamental to relational analytics and ubiquitous in practice. To date, all existing ZK databases systems [26, 32] decompose a k -way join into $k-1$ successive binary joins and supply every intermediate join result as part of the witness to the verification circuit. To prevent privacy leakage through the witness size, which is publicly revealed, these intermediate join results must be padded to their worst-case sizes. This padding compounds multiplicatively across the $k-1$ binary join steps, inflating the circuit size to $O(IN^k)$, where IN denotes the total size of the input database. As discussed above, the proving timescales with circuit size, thus resulting in prohibitive costs in practice. For example, for TPC-H Q3, which involves a 3-way join, PoneglyphDB [26] estimates a proving time of more than 30 years; for Q8, which involves an 8-way join, the estimated proving time exceeds 10^{13} years (these numbers, like all infeasible proving times, are extrapolated from circuit sizes rather than measured; Section 8 details the methodology and assumptions.), far beyond the age of the universe. This naturally leads to a fundamental question:

“Can multi-way joins be verified in zero knowledge without materializing all intermediate results, thereby avoiding the worst-case circuit blowup?”

1.1 Our Results

In this paper, we answer this question affirmatively by presenting VP Join, which supports both acyclic and cyclic multi-way join queries based on a simple yet powerful insight. As discussed earlier, designing a ZK-proof implementation ultimately reduces to constructing an efficient circuit that captures the relationship between the input and the output. For multi-way joins, however, this relationship can be expressed much more efficiently than by explicitly re-executing every binary join step, as existing systems do, which forces all intermediate join results into the witness in the first place. This paradigm shift, from proving-by-re-execution (encoding each binary-join step inside the circuit) to proving-by-correctness-checking (checking only that the prover’s supplied result is correct), eliminates intermediate materialization entirely.

Building on this insight, we first design the Oblivious Join Gate (OBJ), a circuit to verify acyclic joins, i.e., joins whose joining graph admits a tree structure, inspired by classical semi-join reductions along a join tree [9, 47]. The high-level idea is that the prover splits each input table into two groups, clean tuples (those that participate in the join result) and residual tuples (those that do not), and the circuit verifies that this split is correct through several lightweight structural checks. Each check scans the input tables and outputs join results at most a constant number of times, yielding a total circuit complexity linear in the input and output sizes, i.e., $O(IN + OUT)$. Furthermore, we extend OBJ to support projection, predicates, and aggregation on the join result, all without materializing the full intermediate join results. The circuit complexity remains $O(IN + OUT)$, where OUT denotes the size of the final aggregated output rather than the full join result. For cyclic joins, our proposed OBJ does not work since it relies on the existence of a join tree, which requires the query to be acyclic. To address this, we propose the **Tree-Decomposed Join Gate (TDJ)**, which reduces a cyclic query to an acyclic one via tree decomposition [23, 28, 39]: the relations

are grouped into a small number of clusters such that, at the inter-cluster level, each cluster acts as a single “super-relation” and the clusters together form a join tree whose structure is acyclic, so OBJ applies directly. This confines the intermediate blowup from $O(IN^k)$ for the original query to $O(IN^w)$ with one cluster, where w is the tree width [23] (typically $w \leq k$). However, these intra-cluster intermediates are not true input relations but data-dependent materialized results, so their sizes must be hidden to preserve privacy. **By padding each intra-cluster intermediate to its worst-case bound, TDJ hides these sizes completely, so cyclic queries enjoy the same full obliviousness guarantee as acyclic ones. As an optional relaxation for the case where full obliviousness is not required, TDJ can instead set the intra-cluster capacities using differentially private [15] upper bounds, trading a quantified (ϵ, δ) leakage of the intra-cluster capacities for a substantial reduction in padding cost.** In summary, this paper makes the following contributions:

- For acyclic join queries, we propose OBJ, which supports projection, predicates, and aggregation, all without materializing intermediate results. Compared with prior work whose circuit size grows to $O(IN^k + OUT)$, OBJ achieves $O(IN + OUT)$ circuit complexity, where OUT is the size of the final output. For aggregation queries, OUT is the final aggregated output size rather than the full join result. Notably, for aggregation queries, this means our result also improves binary join-aggregate queries, from a circuit complexity of $O(IN^2 + OUT)$ to $O(IN + OUT)$.
- For cyclic joins, we propose **TDJ**, which extends OBJ to support cyclic join verification. First, through tree decomposition, we achieve a circuit complexity of $O(IN^w + OUT)$, where w is the tree width, usually much smaller than k , **while remaining fully oblivious via worst-case intra-cluster padding.** To further reduce the overhead of IN^w caused by padding part of intermediate results, we **additionally provide an optional DP-guided padding mode that achieves** a better privacy–efficiency trade-off.
- We conducted comprehensive experiments on five TPC-H and four graph queries. Experimental evaluations show that VP Join outperforms PoneglyphDB [26] by 6–18 orders of magnitude on TPC-H and 4–9 on graph workloads, reducing Q8 proving time from 10^{13} years to under 200 s.

The remainder of this paper is organized as follows. Section 2 surveys related work. Section 3 introduces preliminaries. Section 4 describes the problem setting. Section 5 presents the ZK proof primitives used throughout. Section 6 details the OBJ gate for acyclic joins. Section 7 extends to cyclic joins via **TDJ, a fully oblivious construction with an optional DP-guided relaxation.** Section 8 reports experimental results.

2 Related Work

Verifiable query processing ensures a result is correctly computed over a committed database; the zero-knowledge (ZK) subset additionally hides the underlying data from the verifier. Purely verifiable (non-ZK) systems based on authenticated data structures [5, 17, 36, 41, 46, 55] and query-authentication systems [7, 40, 51, 54] provide integrity but expose the underlying data. Our work is in the ZK subset, where the prover additionally hides the data. In

this setting, vSQL [52, 53] compiles relational operators into circuits verified via the GKR protocol [21]. ZKSQL [32] decomposes verification into operator-level gadgets, improving modularity but requiring multiple rounds of interaction and coupling the circuit to a fixed query plan. PoneglyphDB [26] builds a practical non-interactive system on Halo2 [49, 50] with PLONKish arithmetization [19], supporting selections, projections, aggregations, sorting, and equi-joins. All of these systems decompose multi-way joins into binary joins, materializing every intermediate inside the circuit. Because circuit sizes are public, intermediate sizes leak cardinality information; padding them to worst-case sizes leads to an $O(IN^k)$ blowup. VPJoin eliminates this problem: its witness-guided verification never materializes intermediates, achieving $O(IN + OUT)$ for acyclic queries, and DP-bounded costs for cyclic queries.

Beyond the ZK setting, SecureYannakakis [45] and SPECIAL [44] use the semijoin reduction strategy in multi-party computation (MPC), but rely on heavy cross-party primitives (PSI [34, 38], garbled circuits [48]) that do not translate efficiently into ZK circuits. Our single-prover model differs fundamentally: it produces publicly verifiable proofs with circuit size as the dominant cost.

3 Preliminaries

3.1 Notations

A database $D = \{R_1, R_2, \dots, R_k\}$ is a finite collection of k relations over a universe of attributes $\mathcal{V} = \bigcup_{i=1}^k \text{attr}(R_i)$, where $\text{attr}(R_i)$ is the schema of R_i and each attribute $A \in \mathcal{V}$ draws values from a finite domain $\text{Dom}(A)$. A tuple over an attributeset $F \subseteq \mathcal{V}$ is an element in $\text{Dom}^F := \prod_{A \in F} \text{Dom}(A)$, and each relation R_i is a finite multiset of tuples over $\text{attr}(R_i)$. For $t \in R_i$ and $F \subseteq \text{attr}(R_i)$, we write $t[F]$ for the restriction of t to F . The cardinality of R_i is denoted $|R_i|$, and the total input size is $IN = \sum_{i=1}^k |R_i|$. Following prior work [26, 32], we assume each $|R_i|$ (and hence $IN = \sum_i |R_i|$) is public.¹

We associate each query with a join hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$, whose vertices are the attributes in $\mathcal{V}$ and whose hyperedges $F_i := \text{attr}(R_i) \in \mathcal{E}$ are the schemas of the input relations. A jointree of $\mathcal{H}$ is a separate tree T (note equal to $\mathcal{H}$) whose nodes are in one-to-one correspondence with the input relations $\{R_1, \dots, R_k\}$ (equivalently, with the hyperedges of $\mathcal{H}$) and which satisfies the running-intersection property [9]: for every attribute $A \in \mathcal{V}$, the nodes whose relations contain A induce a connected subtree of T . A query is acyclic if its hypergraph admits a jointree, and cyclic otherwise. We fix an arbitrary root of T and denote each tree edge as a parent–child pair (P, C) with shared attributes $K_{PC} := \text{attr}(P) \cap \text{attr}(C)$.

The projection of a relation R onto an attributeset $F \subseteq \text{attr}(R)$ is

$$\pi_F(R) = \{t[F] \mid t \in R\},$$

i.e., the set of distinct restrictions of R 's tuples to F . In addition, we use $R \equiv S$ to denote multiset equality: R and S contain the same tuples with the same multiplicities, which is strictly stronger than set equality.

The natural join of $R_1, \dots, R_k$ is the set of tuples over $\mathcal{V}$ whose projection onto each schema $\text{attr}(R_i)$ is a tuple of R_i :

$$J(D) := R_1 \bowtie R_2 \bowtie \dots \bowtie R_k = \{t \in \text{Dom}^{\mathcal{V}} \mid \pi_{\text{attr}(R_i)}(t) \in R_i \text{ for all } i \in [k]\}.$$

Equivalently, two tuples $t \in R_i$ and $s \in R_j$ combine when they agree on their shared attributes $K_{ij} := \text{attr}(R_i) \cap \text{attr}(R_j)$, i.e., $t[K_{ij}] = s[K_{ij}]$.

The semijoin and antijoin, used to isolate matching and non-matching tuples along a single joined edge, are defined for any $i \neq j$ as

$$\begin{aligned} R_i \ltimes R_j &= \{t \in R_i \mid \exists s \in R_j : t[K_{ij}] = s[K_{ij}]\}, \\ R_i \triangleright R_j &= \{t \in R_i \mid \forall s \in R_j : t[K_{ij}] \neq s[K_{ij}]\}. \end{aligned}$$

Finally, we consider aggregation on top of a join, as is common in analytical SQL queries. For a grouping attributeset $G \subseteq \mathcal{V}$ and an aggregation attribute $A \in \mathcal{V}$ (or the wildcard $A = *$, which represents “no attribute” and yields the SQL semantics of $\text{COUNT}(\ast)$), the aggregation operator $\gamma_{G, f(A)}$ groups its input by G and computes an aggregate $f \in \{\text{count}, \text{sum}, \text{avg}, \dots\}$ over A within each group, returning one tuple per distinct group value. Applied to the join result $J(D)$, the join-aggregate query result is

$$J_{\text{agg}}(D) := \gamma_{G, f(A)}(J(D)). \quad (1)$$

If $G = \emptyset$, $J_{\text{agg}}(D)$ collapses to a single scalar.

We use OUT to denote the size of the final query result returned to the verifier, and express all complexity bounds in this paper in IN and OUT . For a plain join, $OUT = |J(D)|$ is the number of join tuples. For a join-aggregate query, $OUT = |J_{\text{agg}}(D)|$ is the number of distinct groups, which can be orders of magnitude smaller than $|J(D)|$ since many jointuples may collapse into the same group. A central feature of our design is that the proof cost scales with the final OUT rather than the full join size $|J(D)|$, even when the two differs substantially.

3.2 Zero-Knowledge Proofs

Zero-knowledge (ZK) proofs [22] allow a prover to convince a verifier that a public statement is true without revealing any additional information beyond the validity of the statement itself. In this work, we use zero-knowledge Succinct Non-interactive Arguments of Knowledge (ZK-SNARKs) [19, 24], which provide three standard guarantees:

- **Completeness.** An honest prover can prove a true statement.
- **Soundness.** A dishonest prover cannot prove a false statement except with negligible probability.
- **Zero-knowledge.** The verifier learns nothing beyond the statement.

At a high level, a ZK-SNARK proves that there exists a private witness satisfying a public set of constraints for a given public statement. The witness consists of the hidden information needed to justify why the statement is true; depending on the application, it may include secret inputs, intermediate values, or an execution trace of the computation being certified. To enable verification, the target computation is encoded as an arithmetic circuit. The circuit is a public object that specifies how the witness and the public statement must relate. In practice, its constraints are represented as low-degree polynomial equations, each of which evaluates to zero if and only if the witness corresponds to a valid execution of the computation. The prover then generates a proof that there exists a

¹If the data size is itself sensitive, the prover can pad each relation with dummy tuples and publish only the padded size.

witness satisfying all these constraints, without revealing the witness itself. This is just an abstract description of a general ZK proof implementation. In the next section, we describe in detail how to implement this process for our problem setting.

4 Problem Definition

In this work, we focus on queries Q consisting of multi-way joins, predicates, projection, and aggregation applied to the join result. Our target is to propose a ZK proof implementation that verifies multi-way join queries in a privacy-preserving manner.

4.1 Workflow

To solve the above problem, we handle each query through the following workflow. Here, we use the ZK-SNARK system, and our target is to prove that the claimed query result was computed correctly from a given input database.

Step 1 Data Commitment: Before answering any queries, the prover publishes a cryptographic commitment using the Inner Product Argument (IPA) protocol [10], $Commit(D)$, which binds all subsequent proofs to a single input database. The details are presented in Appendix A.

Step 2: Proof Generation: When query Q arrives, the prover publishes $Q(D)$. At the same time, the prover generates and publishes a proof showing that the claimed result was derived from the database committed to by $Commit(D)$. This verification is typically not a single-step process. Instead, it often depends on additional intermediate values that help link the committed database to the claimed final output. These intermediate values, together with the private input database, are collectively called the witness. Importantly, the witness must remain hidden from the verifier. To make verification possible, the intended computation is encoded as an arithmetic circuit which specifies the relationships among the witness, the committed database, and the claimed result. The prover then generates a proof that there exists a private witness such that all circuit constraints are satisfied with respect to the committed database and the claimed output, without revealing the witness itself. In practice, these constraints are represented as low-degree polynomial equations, each of which evaluates to zero if and only if the witness corresponds to a correct execution of the computation that produces the claimed result. This can be done by running an existing ZK-SNARK system, specifically Halo2 [49], over the designed circuit and witness.

Step 3: Proof Verification: By checking that these equations are satisfied, the verifier can efficiently validate the correctness of the claimed result with logarithmic running time.

4.2 Threat Model and Security Guarantees

We consider a mutually distrustful setting between the prover and the verifier, which gives rise to three threats: (i) the prover may answer queries using a fabricated or modified database rather than the one it originally committed to; (ii) the prover uses the committed database but returns an incorrect query result; and (iii) a curious verifier may attempt to infer information about the private database beyond what is intentionally revealed by the query result.

Threat (i) is addressed by the database commitment: every subsequent proof is generated with respect to $Commit(D)$, preventing

the prover from silently switching to a different database without causing verification to fail. Threat (ii) is addressed by the standard security properties of ZK-SNARKs, which are inherited by our Halo2 [49] instantiation [11, 50]. To defend against Threat (iii), since the circuit is publicly visible, we require it to be oblivious, meaning that its size and structure should not leak any information about the database. Under this condition, once the proof is generated by ZK-SNARKs (Halo2 in our implementation), the zero-knowledge property of ZK-SNARKs [22] guarantees that the proof will not leak any information besides the query results.

4.3 Problem Reduction

We now analyze the running time of the above workflow. The prover can compute the data commitment (Step 1) in linear time [19], and the verifier can verify the proof (Step 3) in logarithmic time [12]; see Appendix A for the details. As a result, the main bottleneck lies in proof generation (Step 2). In this step, when the prover uses Halo2 [49] to generate a proof over the circuit and witness, the running time is roughly linear in the circuit size. In this paper, we treat the underlying cryptographic procedures of generating proofs from circuits as black boxes and defer the details to Appendix A. As a result, the remaining task is to design a circuit that is (1) oblivious and (2) efficiently encodes the relationship between the input and the claimed output.

Designing such a circuit for multi-way joins, however, is non-trivial. Existing ZK database systems decompose a multi-way join into a sequence of binary joins and include every intermediate join result in the witness. Because each witness value must occupy circuit cells and be enforced by constraints, witness size and circuit size are tightly coupled. Therefore, reducing proving time requires reducing the witness size as well, which turns the problem into designing an oblivious circuit that avoids materializing large intermediate join results.

5 ZK Proof Primitives

With the problem setting and privacy model established, we now describe the ZK proof primitives that serve as building blocks for our join-verification circuits. Our cryptographic backend is Halo2, whose setup and proof-complexity model are detailed in Appendix A.

5.1 PLONKish Arithmetization

We briefly review PLONKish arithmetization; see [26] for details and Appendix B for an example. A PLONKish circuit is a rectangular matrix with three column types: fixed columns, which store circuit-defined constants such as selectors; advice columns, which hold private witness values assigned by the prover; and instance columns, which contain public inputs and outputs. Because our join-verification logic mainly uses advice columns, we write columns to mean advice columns unless stated otherwise.

Computation is enforced by two kinds of constraints. Gate constraints are low-degree polynomials over the columns that must vanish on each row; selector columns activated different gate types on different rows, allowing heterogeneous operations in one circuit. Copy constraints enforce equality across cells via a permutation argument [19], linking output of one gate to input of another. The

prover assigns the advice columns to form an execution trace, and the proof certifies that all constraints hold without revealing the trace.

We use a gate as a slightly broader sense than a single constrained row: a gate is a reusable sub-circuit implementing one logical operation, such as a range check, lookup, or semijoin check. Larger circuits are built by composing such gates, and total circuit size is the sum of the costs of all instantiated gates.

5.2 Lookup Arguments and Derived Gates

Lookup arguments [18, 27] allow proving that every element of a private witness column P belongs to a public table Q . In this paper, we use the Plookup protocol [18], which verifies permutation integrity via a grand product argument:

$$\prod_i (P_i + \alpha)(Q_i + \beta) = \prod_i (P'_i + \alpha)(Q'_i + \beta) \quad (2)$$

where P', Q' are sorted permutations of P and Q , i ranges over all row indices, and α, β are random challenges.

R1.01 Why Eq. (2) certifies a secret permutation in $O(n)$. At first glance, Eq. (2) may look ill-formed as a permutation predicate: the row index i appears on both sides, yet a permutation maps row i to a different row $j = \sigma(i)$, and explicitly representing a secret permutation σ seemingly requires $\Theta(n \log n)$ information. The key observation is that Eq. (2) is not a pointwise identity between rows: each side is a grand product over all rows and is therefore invariant to any reordering of its factors. The differing indices are thus exactly what makes it a multiset check—the equation asserts only that the two sides are equal as multisets, a statement independent of the particular correspondence σ . Two size- n multisets P and P' are equal if and only if $\prod_i (X + P_i) = \prod_i (X + P'_i)$ holds as a polynomial identity in X ; by the Schwartz–Zippel lemma, checking this identity at a random challenge point fails with probability at most $n/|F|$, which is negligible over a 255-bit field. This is the standard shuffle/permutation argument [8, 19, 35], a widely used primitive rather than one specific to our work or to PoneglyphDB [26], whose permutation equation is an instance of the same identity: the prior ZK database systems vSQL and ZKSQL [32, 52] prove permutations with the same random-evaluation polynomial identity, which also underlies PLONK [19], Plookup [18], and Halo2's lookup argument [49]. In particular, vSQL [52] makes the same distinction for join/set-intersection verification, contrasting routing-based permutation checking with an algebraic product-identity check for multiset equality. Why the permutation stays secret at no extra cost. The reason no $O(n \log n)$ blowup arises is specific to the ZK-SNARK setting: the circuit never computes the reordering, it only verifies consistency between prover-supplied columns. The prover computes the re-ordered column P' off-circuit and supplies it as a private witness; this secret permutation σ itself never enters the witness, because the multiset identity above does not mention σ . This is fundamentally different from secure-computation models such as MPC, where no single party sees all the data and the function must therefore be computed obliviously in-circuit; there, applying a secret permutation indeed requires routing the data through a switching network with $O(n \log n)$ switches. Generating P' in-circuit would incur the same cost, but merely linking P and P' requires only the $O(n)$ multiset check above. Moreover, since all witness columns (including P and

P') are hidden by the zero-knowledge property, no proof—nor any sequence of proofs over the same data—reveals σ or any intermediate cardinality: each proof exposes only the public statement it certifies.

From permutation to subset. The subset predicate needed by lookups—that every value of P appears in the table Q —adds only one constraint per row on top of the permutation argument, exactly as in the lookup argument of four Halo2 backends [49]: the prover supplies P' with equal values adjacent (e.g., sorted) and Q' as a permutation of Q aligned to P' , so that the first row of each run of equal values in P' holds a matching table entry (i.e., $Q'_i = P'_i$ there, with the unused table entries filling the remaining rows). The circuit then enforces the boundary condition $P'_0 = Q'_0$ and, for every row $i > 0$, the local constraint $(P'_i - Q'_i)(P'_i - P'_{i-1}) = 0$, which forces each P'_i to equal either its aligned entry Q'_i or its predecessor P'_{i-1} . By induction over the runs of equal values, every value of P' (hence of P) must appear in Q' (hence in Q), establishing the subset relation at $O(1)$ cost per row.

For large domains (e.g., 64-bit integers), a decomposed strategy splits each value into $tot = \lceil B/b \rceil$ chunks of b bits and checks each chunk against a small table of size 2^b , making the number of constraints independent of the domain size 2^B .

Building on lookup arguments, we employ two derived gates used throughout this work:

- RangeCheck [25, 26]. Verifies that a value in a column lies within $[0, 2^B)$ using the decomposed lookup strategy described above. Each value is split into $tot = \lceil B/b \rceil$ chunks of b bits, and each chunk is checked against a lookup table of size 2^b .
- Sort [26]. Verifies that the values in a column are in non-decreasing order by computing consecutive differences $d_i = x_{i+1} - x_i$ and applying a range check to enforce $d_i \geq 0$.

These primitives, together with the Plookup and permutation arguments, serve as the foundation for the Oblivious Join Gate presented in Section 6.

6 OBJ: Efficient Verification for Acyclic Joins

The core challenge for join verification is to certify the query result without materializing any intermediate join inside the circuit, since each materialized intermediate must be padded to its worst-case bound to achieve obliviousness, leading to prohibitive circuit sizes of $O(IN^k)$ for a k -way join. Our insight is that an acyclic multi-way join can be decomposed via semijoin reductions along a join tree, and each reduction can be verified independently with a linear-cost circuit check. Rather than re-executing the join, the prover splits each input table into clean tuples (those participating in the join result) and residual tuples (those that do not), and the circuit verifies this split through lightweight structural checks without materializing intermediate join results.

In this section, we propose Oblivious Join Gate (OBJ), which achieves efficient verification for acyclic joins, and build it up from the local building block to the full query. We begin in Section 6.1 with a semijoin gate: for a single edge (R_i, R_j) of the join tree, it certifies that the prover has correctly separated R_i into clean and residual tuples with respect to R_j . This gate has linear cost $O(IN)$ and serves as the basic module for the rest of the section. Building on the semijoin gate, we then propose the full OBJ gate with

two variants: a direct Two-Pass OBJ that is similar to the classical bottom-up/top-down semijoin reduction [9, 47] (Section 6.2.1), and a more efficient One-Pass OBJ in which the prover directly supplies the final clean/residuals split so the circuit only has to certify it rather than reconstruct the reduction in-circuit (Section 6.2.2). Section 6.3 extends the same clean-relation representation to aggregation by propagating tuple multiplicities while still achieving $O(IN + OUT)$ cost, and we handle projection and predicates under the same interface. **Section 6.4 adds a final output-certification step that pins down the reported output multiset for plain joins.**

R3.05

6.1 Semijoin Gate

We start with the fundamental building block of OBJ: a gate that certifies, for a single join-tree edge, that the prover's claimed partition of a relation into clean and residual tuples is correct. Once this linear-cost gate is available, the full OBJ is obtained by composing one instance per tree edge.

Consider one edge (R_i, R_j) of a joint tree with shared join attributes $K_{ij} = \text{attr}(R_i) \cap \text{attr}(R_j)$. The prover claims that two witness tables R'_i and R''_i realize the exact semijoin/antijoin split of R_i with respect to R_j :

$$R'_i = R_i \ltimes R_j, \quad R''_i = R_i \triangleright R_j. \quad (3)$$

The gates should certify this claim without recomputing the semijoin inside the circuit. We specify each gate in this paper by three factors: the private witness (what secret data the prover supplies), the output (what the gate certifies for downstream use within the circuit), and the verification target (what property the circuit constraints enforce).

Definition 1 (Semijoin Gate). Fix one directed edge (R_i, R_j) of a joint tree T and its shared join attributes K_{ij} . We define the semijoin gate $\text{SJ}_{i \rightarrow j}$ through the following three factors:

- Private witness. The tuples of R'_i and R''_i , together with the prover-supplied partition (R'_i, R''_i) .
- Output. A certified partition (R'_i, R''_i) , where R'_i is the kept part and R''_i is the residual part. This output is usually consumed by later gates rather than exposed directly to the verifier.
- Verification target. The gate certifies (3), i.e., that (R'_i, R''_i) is exactly the semijoin/antijoin split of R_i with respect to R_j . Equivalently, the witness must satisfy three semantic conditions: (i) conservation of R_i across (R'_i, R''_i) , (ii) membership of R'_i 's keys in R_j , and (iii) non-membership of R''_i 's keys in R_j , formalized respectively as:

$$R_i \equiv R'_i \cup R''_i, \quad (4)$$

$$\pi_{K_{ij}}(R'_i) \subseteq \pi_{K_{ij}}(R_j), \quad (5)$$

$$\pi_{K_{ij}}(R''_i) \cap \pi_{K_{ij}}(R_j) = \emptyset. \quad (6)$$

Design idea. Our target is to certify $R'_i = R_i \ltimes R_j$ and $R''_i = R_i \triangleright R_j$ without ever computing the semijoin inside the circuit. Our insight is that these two equalities jointly decompose into three weaker structural conditions that together imply them: conservation of R_i (no tuples lost or fabricated), membership of R'_i 's keys in R_j , and non-membership of R''_i 's keys in R_j . Each of these is linear-cost to check, and together they leave no cheating strategy: a joinable tuple hidden in R''_i would violate non-membership, a fabricated tuple in R'_i would violate conservation, and a non-joinable tuple in R'_i would

violate membership. The gate therefore reduces two quadratic-looking equality checks to three linear-cost structural checks.

Realization. The prover locally computes the partition (R'_i, R''_i) and supplies it directly as witness columns. Since the partition is exhaustive, the combined size $|R'_i| + |R''_i| = |R_i|$ is determined solely by the public input size. The circuit then enforces the three verification conditions (4), (5), and (6) from the Verification target of Definition 1 using three checks.

- Conservation Check (permutation argument) realizes (4). By treating the concatenation $R'_i \cup R''_i$ as a single virtual column and checking via the permutation argument ((2)) that it is a permutation of R_i , the circuit enforces multiset equality, guaranteeing that no tuples are fabricated or dropped. This check contributes $O(IN)$ constraints.
- Membership Check (lookup argument) realizes (5). Treating the key column of R'_i as the witness and the key column of R_j as the reference table, the lookup argument enforces $\pi_{K_{ij}}(R'_i) \subseteq \pi_{K_{ij}}(R_j)$, proving that every kept tuple has a matching key in R_j . This check contributes $O(IN)$ constraints.
- Non-Membership Check (merge-and-sort) realizes (6). A lookup argument cannot directly prove that a value is not in a table, so we use a merge-and-sort strategy: (1) the prover merges both key columns into one and provides a sorted permutation of the merged sequence as witness; (2) a permutation argument verifies that the sorted sequence is a rearrangement of the merged keys; (3) a sort gate verifies non-decreasing order; and (4) a local gate constraint checks that no two adjacent elements from different sources are equal. If the sorted merged sequence has no equal adjacent pair from the two sources, the key sets must be disjoint. This check contributes $O(IN)$ constraints.

Since lookups and permutation arguments operate on single-column values, multi-attribute tuples are first compressed to one field element per tuple. Putting the pieces together, the semijoin gate has linear cost $O(IN)$: the witness layout allocates $|R_i|$ rows at no constraint cost, and each of (4), (5), (6) contributes $O(IN)$ constraints. The detailed constant-factor breakdown appears in Appendix F.

6.2 From Semijoin Verification to OBJ

With the semijoin gate in hand, we can now move from local verification to whole-query verification. The goal is no longer just to certify one local split $R_i = R'_i \cup R''_i$ on a single edge, but to certify that the collection of all reduced (“clean”) relations across the whole join tree forms the fully reduced instance of the acyclic query. OBJ accomplishes this by arranging semijoin checks along a joint tree and treating the clean relations as a compact witness for the final result.

Definition 2 (Oblivious Join Gate (OBJ)). Fix an acyclic join query together with a joint tree T . We define the Oblivious Join Gate through the following three factors:

- Private witness. The input relations $\mathcal{R} = \{R_1, \dots, R_k\}$, together with prover-supplied auxiliary data. We will present two realizations of the OBJ, each with its own witness layout, so the detailed witness is left abstract here and specified per realization.
- Output. The certified clean instance $\mathcal{R}^c = \{R_1^c, \dots, R_k^c\}$, where each $R_i^c \subseteq R_i$ contains exactly the tuples of R_i that participate in the global join; by complement, the residual relations are

$\mathcal{R}^r = \{R_1^r, \dots, R_k^r\}$ with $R_i^r = R_i \setminus R_i^c$. As with the semijoin gate, $\mathcal{R}^c$ is consumed within the circuit: the final join result or aggregate is computed from $\mathcal{R}^c$ after the gate has certified it.

- Verification target. The gate certifies that $\mathcal{R}^c$ is the fully reduced instance of the acyclic query: it removes every dangling tuple and keeps exactly those tuples that can be extended to at least one global join result. Since we will present two realizations of the OBJ, each with its own set of concrete verification conditions, we specify these conditions per realization in Sections 6.2.1 (Two-Pass OBJ) and 6.2.2 (One-Pass OBJ) rather than here.

The definition fixes only the input/output contract: given $\mathcal{R}$, the gate must output a clean instance $\mathcal{R}^c$ that equals the fully reduced instance of the query. It does not prescribe how $\mathcal{R}^c$ is produced or how many semijoin gates are invoked internally; any circuit that certifies this contract qualifies as an OBJ realization. Semijoin gates supply the local edge-level checks, and the realization chooses how to compose them along T . The next two subsections present two such realizations: a direct Two-Pass OBJ that closely follows the classical bottom-up/top-down semijoin reduction along the join tree T , and a more efficient One-Pass OBJ that takes advantage of prover-supplied final clean/residual partitions.

6.2.1 Two-Pass OBJ. We start from the classical semijoin reduction [47] for acyclic queries. Given a join tree T with an arbitrary root, the algorithm performs two in-place semijoin passes: a bottom-up pass that, for each edge (P, C) processed from leaves to root, replaces the parent by $P \leftarrow P \bowtie C$, so that after the pass the root retains only tuples with a matching combination in every descendant subtree; and a top-down pass that, for each edge (P, C) processed from root to leaves, replaces the child by $C \leftarrow C \bowtie P$ on the already-reduced parent, so that every relation retains only tuples extendable to at least one global join result. We adapt this algorithm to the ZK setting by replacing each in-place semijoin with one invocation of the semijoin gate of Section 6.1. Concretely: (i) for every tree edge we invoke a semijoin gate in the bottom-up pass and another in the top-down pass, yielding $2(k-1)$ gate invocations that jointly certify the clean relations $\{R_1^c, \dots, R_k^c\}$; and (ii) a final pairwise consistency check across all edges certifies that the clean relations agree on their shared keys, ruling out the corner case where two edges are each individually consistent but jointly inconsistent. The circuit never recomputes a semijoin; it only verifies prover-supplied partitions edge by edge. Realization. We compose semijoin gates along all edges of the join tree T to verify the full acyclic join $R_1 \bowtie \dots \bowtie R_k$ in two passes (bottom-up and top-down).

In the bottom-up pass, we process edges from leaves to root, applying a semijoin gate for each edge (P, C) to reduce the parent: $P \leftarrow P \bowtie C$. After processing all children of a node, that node retains only tuples that have a match in every descendant subtree. For instance, in a tree rooted at R_2 with children R_1 and R_3 (as in Example 6.1), the bottom-up pass reduces R_2 by R_1 and then by R_3 , yielding the reduced root R_2^c .

In the top-down pass, starting from the fully reduced root, we propagate the reduction outward. For each edge (P, C) processed from root to leaves, we apply a semijoin gate to reduce the child: $C \leftarrow C \bowtie P$, where P is the already-reduced parent. This filters each relation by its reduced parent, ensuring global consistency across all tree edges. Across the two passes, every input relation R_i is split into the final

clean part R_i^c and the global residual $R_i^r = R_i \setminus R_i^c$; R_i^r aggregates all tuples of R_i discarded by the semijoin gates on edges incident to R_i .

After both passes produce the fully reduced relations $\{R_1^c, R_2^c, \dots, R_k^c\}$, a pairwise consistency check verifies that adjacent clean relations agree on their shared join keys. For every edge (R_i, R_j) with shared attributes K_{ij} , the circuit checks $\pi_{K_{ij}}(R_i^c) = \pi_{K_{ij}}(R_j^c)$ via two Membership Checks ($R_i^c \rightarrow R_j^c$ and $R_j^c \rightarrow R_i^c$). This certifies that every clean tuple has a matching join partner in every adjacent clean relation, ensuring the reduced instance admits no dangling tuples.

Finally, the output is computed by joining the verified clean relations along the tree: **since pairwise consistency guarantees key agreement across every edge, the prover can enumerate the join indirectly from the reduced relations. This enumeration itself, however, is a prover-side step: certifying that the reported output equals this join—with no row misreported, duplicated, or omitted—requires the additional output-certification step of Section 6.4.**

We defer the detailed correctness and obliviousness analyses of both OBJ variants to Appendix E.

Example 6.1 (Two-Pass OBJ on a 3-way path join). Consider the acyclic join $Q = R_1(A, B) \bowtie R_2(B, C) \bowtie R_3(C, D)$, with join predicates $R_1.B = R_2.B$ and $R_2.C = R_3.C$ (as visualized in Figure 1). Let the input relations be R_1, R_2 and R_3 .

With the join tree rooted at R_2 (so R_1 and R_3 are its children), we begin with the bottom-up pass to reduce each parent relation against its children along tree edges, keeping only tuples whose join keys have a matching combination in every descendant subtree. For each edge, the prover supplies a kept witness $R_i \bowtie R_j$ and a residual witness $R_i \triangleright R_j$, and the circuit enforces conservation via a permutation argument, membership for kept keys via lookup, and non-membership for residual keys. Specifically, the following two steps are performed.

Step 1: Process edge $(P, C) = (R_2, R_1)$, reducing parent R_2 against child R_1 on key B . Since $\pi_B(R_1) = \{1, 2, 3, 6\}$, the tuples $(4, 14)$ and $(5, 16) \in R_2$ (with $B = 4, 5$) cannot participate in the join with R_1 and are discarded. The semijoin gate partitions $R_2 = R_2' \cup R_2''$:

$$R_2' = R_2 \bowtie R_1 = \{(1, 11), (2, 12), (3, 13)\}, \quad R_2'' = R_2 \triangleright R_1 = \{(4, 14), (5, 16)\}.$$

The circuit checks: (1) $R_2 \equiv R_2' \cup R_2''$ (conservation via permutation), (2) $\pi_B(R_2') \subseteq \pi_B(R_1)$ (membership via lookup), (3) $\pi_B(R_2'') \cap \pi_B(R_1) = \emptyset$ (non-membership). These three checks are performed similarly at the end of the next three steps and are omitted thereafter for brevity.

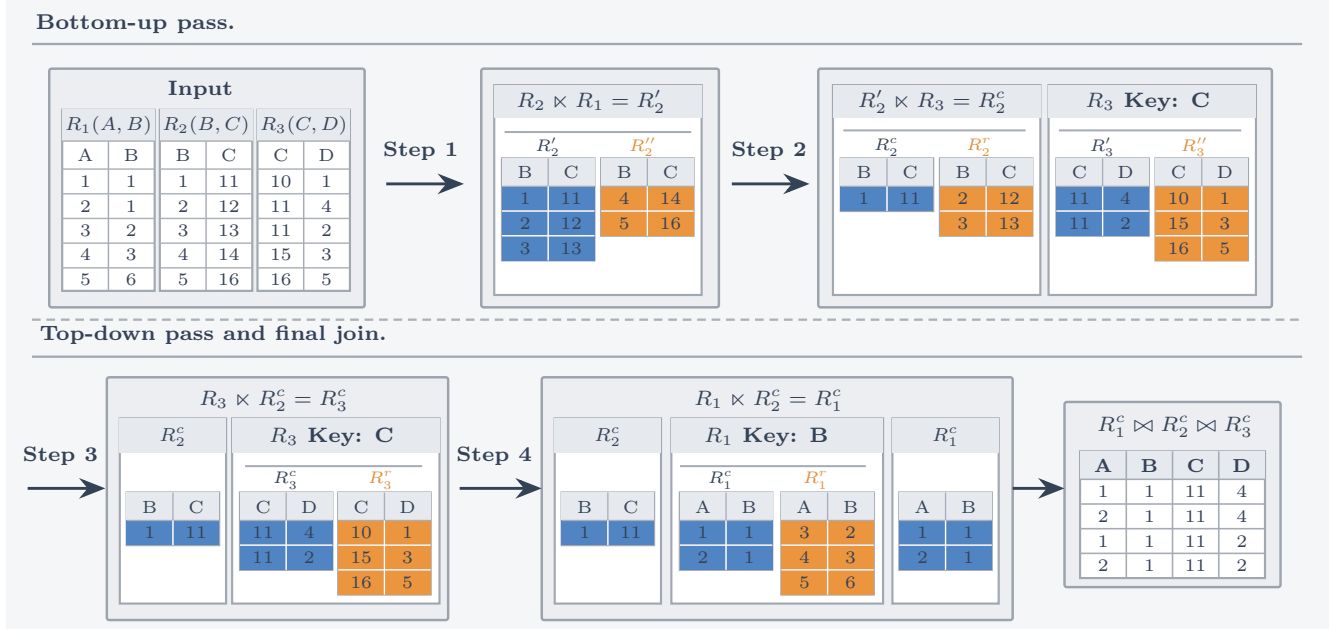
Step 2: Process edge $(P, C) = (R_2, R_3)$, reducing parent R_2' against child R_3 on key C . Now $\pi_C(R_2') = \{11, 12, 13\}$ and $\pi_C(R_3) = \{10, 11, 15, 16\}$ intersect only on $C = 11$, so R_2' is split into $R_2^c = R_2' \bowtie R_3 = \{(1, 11)\}$ and a local discard $R_2' \triangleright R_3 = \{(2, 12), (3, 13)\}$. Aggregating with Step 1 gives the global residual for R_2 :

$$R_2^r = R_2 \setminus R_2^c = R_2'' \cup (R_2' \triangleright R_3) = \{(2, 12), (3, 13), (4, 14), (5, 16)\}.$$

After this step the root R_2^c is fully reduced, while the children R_1 and R_3 remain untouched; the top-down pass will reduce them against R_2^c .

With R_2^c fully reduced, we begin the top-down pass to propagate its keys outward and reduce each child against the already-reduced parent, so that every remaining child tuple can be extended to at least one global join result. This includes the following Steps 3 and 4.

Step 3: Process edge $(P, C) = (R_2, R_3)$ (top-down), reducing child R_3 against reduced parent R_2^c on key C . Since $\pi_C(R_2^c) = \{11\}$, only tuples

Figure 1: Illustration of the Two-Pass OBJ on the SQL query $\text{SELECT } * \text{ FROM } R_1, R_2, R_3 \text{ WHERE } R_1.B = R_2.B \text{ AND } R_2.C = R_3.C$.

in R_3 with $C = 11$ survive:

$$R_3^c = R_3 \times R_2^c = \{(11, 4), (11, 2)\}, \quad R_3^r = R_3 \triangleright R_2^c = \{(10, 1), (15, 3), (16, 5)\}.$$

Step 4: Process edge $(P, C) = (R_2, R_1)$ (top-down), reducing child R_1 against R_2^c on key B . Since $\pi_B(R_2^c) = \{1\}$, only tuples in R_1 with $B = 1$ survive:

$$R_1^c = R_1 \times R_2^c = \{(1, 1), (2, 1)\}, \quad R_1^r = R_1 \triangleright R_2^c = \{(3, 2), (4, 3), (5, 6)\}.$$

After the two passes, the fully reduced (clean) instance is:

$$R_1^c = \{(1, 1), (2, 1)\}, \quad R_2^c = \{(1, 11)\}, \quad R_3^c = \{(11, 4), (11, 2)\}.$$

R3.O5 As the final step of the Two-Pass OBJ, the circuit verifies edge consistency via mutual lookups: $\pi_B(R_1^c) = \{1\} = \pi_B(R_2^c)$ and $\pi_C(R_2^c) = \{11\} = \pi_C(R_3^c)$. At this point $\{R_1^c, R_2^c, R_3^c\}$ is certified as the fully reduced instance of Q , and the final join is evaluated directly from it: $Q(D) = R_1^c \bowtie R_2^c \bowtie R_3^c$, yielding $|Q(D)| = 2 \cdot 1 \cdot 2 = 4$ tuples, matching Figure 1. The reported output is then certified against the clean relations as described in Section 6.4. ■

6.2.2 One-Pass OBJ. The Two-Pass OBJ reconstructs the reduced instance inside the circuit by invoking $2(k-1)$ semijoin gates, traversing every tree edge once bottom-up and once top-down. Our insight is that in a non-interactive proof the prover already knows the final answer before the circuit runs, so the reconstruction can be done outside the circuit: the prover computes the clean/residual split $(\mathcal{R}^c, \mathcal{R}^r)$ off-circuit using any efficient method (for instance the classical two-pass semijoin reduction on the database side) and supplies it directly as a witness. The circuit's task therefore shrinks from producing the reduction to certifying a prover-supplied one.

Certifying a prover-supplied partition reduces to three intuitive questions: (i) Does (R_i^c, R_i^r) actually partition R_i , with no tuple fabricated, lost, or double-counted? (ii) Is the clean side a coherent join skeleton, i.e., does every clean tuple have a matching join partner in every adjacent clean relation? (iii) Was any joinable tuple hidden

in the residual side? Questions (i) and (ii) decompose into local per-relation and per-edge checks. Question (iii) looks harder because a hidden joinable tuple is a global witness, but it reduces to a single lightweight pass: any such tuple would survive as a semijoin reduction over $\mathcal{R}^r$ along the tree, so it suffices to verify that this reduction empties out. Together, these three properties replace the full in-circuit top-down pass of the Two-Pass variant with cheaper per-relation disjointness checks and a single residual-side pass.

Realization. The prover supplies $\mathcal{R}^c$ and $\mathcal{R}^r$ directly as witness columns, and the circuit certifies $\mathcal{R}^c$ as the fully reduced instance by enforcing four conditions:

$$R_i \equiv R_i^c \cup R_i^r, \quad \forall i \in [k], \quad (7)$$

$$R_i^c \cap R_i^r = \emptyset, \quad \forall i \in [k], \quad (8)$$

$$\pi_{K_{ij}}(R_i^c) = \pi_{K_{ij}}(R_j^c), \quad \forall (R_i, R_j) \in T, \quad (9)$$

$$R_k^c \times (R_{k-1}^r \times (\dots (R_2^r \times R_1^r) \dots)) = \emptyset. \quad (10)$$

We describe how to realize the circuit for these four conditions.

- **Conservation Check.** (7) requires the witness to cover R_i exhaustively: no tuple may be fabricated or dropped. It is realized by a single Conservation Check (permutation argument) between R_i and the concatenation $R_i^c \cup R_i^r$ treated as a single virtual column.
- **Non-Membership Check.** (7) alone does not prevent the prover from placing the same tuple in both R_i^c and R_i^r ; such duplication would let her later hide a joinable tuple in the residual while still accounting for it on the clean side. (8) closes this gap by forcing the two sides to be disjoint, realized by one Non-Membership Check per relation (k checks total).
- **Pairwise Consistency Check.** After (7) and (8), $(\mathcal{R}^c, \mathcal{R}^r)$ is a partition of $\mathcal{R}$, but nothing yet forces $\mathcal{R}^c$ to be a coherent join skeleton: a clean tuple in R_i^c could have no matching clean partner in an adjacent R_j^c . (9) closes this gap by requiring the clean sides to agree on every tree

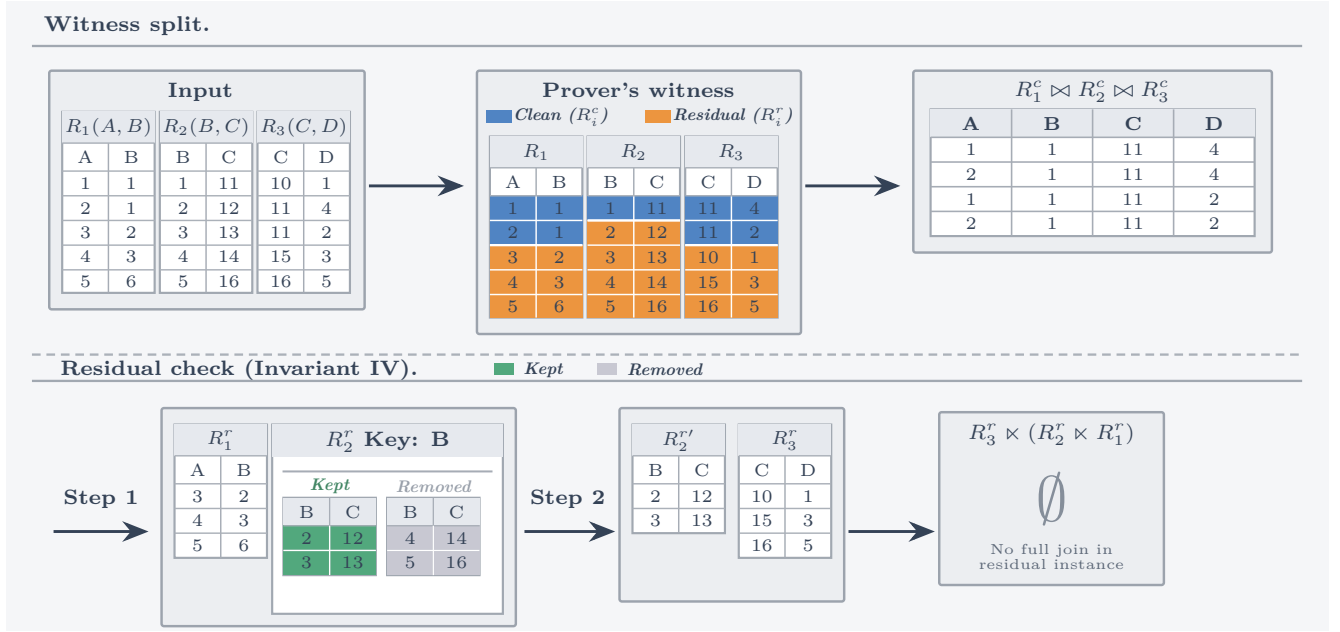


Figure 2: Illustration of the One-Pass OBJ on the SQL query $\text{SELECT}^* \text{FROM } R_1, R_2, R_3 \text{ WHERE } R_1.B = R_2.B \text{ AND } R_2.C = R_3.C$.

edgeto project to the same keyset, so every clean tuple is extendable along each adjacent edge. Each edge's set equality is realized by two mutual Membership Checks ($R_i^c \rightarrow R_j^c$ and $R_j^c \rightarrow R_i^c$) on K_{ij} .

- **Residual Disconnectedness Check.** Even with (7)–(9), a valid joining tuple could still be hidden in $\mathcal{R}^r$, as the clean-side checks say nothing about the residual. (10) closes this last gap: any such hidden tuple would survive a bottom-up semi-join reduction on $\mathcal{R}^r$ along the join tree T , so it suffices to verify that this reduction empties at the root. It is realized by $k-1$ semi-join gates, one per tree edge, in a single bottom-up pass.

R3.O5 Together, (7)–(10) certify $\{R_1^c, \dots, R_k^c\}$ as the fully reduced instance of the query; the prover then computes the final join directly from these clean relations, and the circuit certifies the reported output against them via the output-certification step of Section 6.4 (for plain joins) or the aggregation machinery of Section 6.3 (for join-aggregates). A detailed correctness proof is given in Appendix E alongside the obliviousness analysis.

Example 6.2 (One-Pass OBJ on a 3-way path join). Consider the same query and data as in Example 6.1 (Figure 2). With the join tree rooted at R_2 (so R_1 and R_3 are its children on edges (R_2, R_1) and (R_2, R_3)), the prover supplies the full clean/residual split directly as witness: $R_1^c, R_1^r, R_2^c, R_2^r, R_3^c, R_3^r$. The circuit then certifies this witness by enforcing the four conditions (7)–(10) in a single pass. Specifically, the following four steps are performed.

Step 1. For each relation, the circuit checks (7), $R_i \equiv R_i^c \cup R_i^r$, via a permutation argument, for instance

$$R_2 \equiv \{(1, 11)\} \cup \{(2, 12), (3, 13), (4, 14), (5, 16)\},$$

so no tuple of R_2 is fabricated, dropped, or duplicated. **Step 2.** For each relation, the circuit verifies (8), $R_i^c \cap R_i^r = \emptyset$, via one Non-Membership Check, for instance

$$\{(1, 1), (2, 1)\} \cap \{(3, 2), (4, 3), (5, 6)\} = \emptyset.$$

Step 3. (9) demands that on every tree edge the two clean sides agree on the shared join key. Using mutual Membership Checks, the circuit verifies this on each of the two edges:

$$\pi_B(R_1^c) = \{1\} = \pi_B(R_2^c) \text{ on edge } (R_2, R_1),$$

$$\pi_C(R_2^c) = \{11\} = \pi_C(R_3^c) \text{ on edge } (R_2, R_3),$$

so every clean tuple has a matching partner in every adjacent clean relation.

Step 4. Finally, (10) requires a bottom-up semi-join reduction on the residual relations along the tree, from leaves to root, that empties at the root. Starting from the leaf R_1^r on edge (R_2, R_1) , the parent is reduced: $R_2^r \bowtie R_1^r$. The tuples $(4, 14), (5, 16) \in R_2^r$ have $B \in \{4, 5\} \notin \pi_B(R_1^r) = \{2, 3, 6\}$ and are removed, leaving $\{(2, 12), (3, 13)\}$ with $\pi_C = \{12, 13\}$. Next, on edge (R_2, R_3) the partial result is reduced against the other leaf R_3^r : since $\pi_C(R_3^r) = \{10, 15, 16\}$ is disjoint from $\{12, 13\}$, the root semi-join is empty, certifying that no joinable tuple is hidden in the residuals.

Since all four conditions hold, $\{R_1^c, R_2^c, R_3^c\}$ is certified as the fully reduced instance of Q , and the final join is evaluated directly: $Q(D) = R_1^c \bowtie R_2^c \bowtie R_3^c$, yielding $|Q(D)| = 2 \cdot 1 \cdot 2 = 4$ tuples, matching Figure 2. The reported output is then certified against the clean relations as described in Section 6.4. ■ **R3.O5**

Complexity Analysis. Both OBJ variants achieve $O(IN + OUT)$ circuit complexity. They differ only in their constant factors, which arise from the number and type of underlying primitives.

- **Two-Pass OBJ:** $2(k-1)$ semi-join gates (one per tree edge in each of the bottom-up and top-down passes), plus $2(k-1)$ Membership Checks for pairwise consistency.
- **One-Pass OBJ:** k Conservation Checks, k Non-Membership Checks, $2(k-1)$ Membership Checks for pairwise consistency, and $k-1$ semi-join gates for residual disconnectedness.

The One-Pass OBJ has a smaller constant factor because it replaces the Two-Pass top-down pass ($k-1$ full semi-join gates) with k lightweight Conservation and Non-Membership Checks. Detailed per-gate cost derivations, total constant-factor comparisons, and the savings under uniform sizes appear in Appendix F.

6.3 Aggregation with One-Pass OBJ

Once the One-Pass OBJ certifies the clean relations, those relations become an efficient input for downstream operators, allowing them to operate on the certified $\mathcal{R}^c$ with cost proportional to $|\mathcal{R}^c| \leq IN$ rather than to the full join size $|J(D)|$. We first show this for aggregation. Rather than materializing the full join result to compute aggregates, the prover attaches to each clean tuple an auxiliary count recording how many join rows the tuple participates in, and the circuit verifies these counts bottom-up along the join tree. The final aggregate is then computed as a single pass over the root relation's annotated tuples. For example, $\text{count}(\ast)$ reduces to summing these counts across the clean tuples of the root relation, which equals the total join size without ever enumerating the join. The same idea also yields sum and avg .

Realization. Now that the One-Pass OBJ produces the set of clean relations $\mathcal{R}^c = \{R_1^c, \dots, R_k^c\}$, the task is to compute the join-aggregate $J_{\text{agg}}(D) = \gamma_{G; f(A)}(J(D))$ (Equation (1)) without materializing $J(D)$. We focus on the common case of free-connex acyclic queries [6], i.e., those whose output attributes form a connected subset within the join tree; non-free-connex queries can be reduced to this case with an additional projection gate on the output attributes before aggregation.

The key technical idea is to propagate an auxiliary per-tuple count along the join tree, following the annotated-relations paradigm [4]. Propagation is needed because the count we ultimately want at a root tuple (the number of join rows that extend that tuple through the rest of the tree) is a global quantity: it depends on how many tuples in every descendant subtree match the root tuple on shared keys, and cannot be read off from the root tuple alone. Propagation computes this count bottom-up, so each parent's count is assembled as a product-of-sums over its children's already-established counts, without ever enumerating the join.

Tuple Multiplicities. We call this per-tuple count the multiplicity of a tuple and denote it $\mu(t)$, so $\mu(t)$ is the number of global join rows to which t contributes. The prover supplies $\mu(t)$ for every tuple of every clean relation as part of the auxiliary witness. Leaf-relation multiplicities are trivially 1, and internal-node multiplicities encode the number of join extensions into the subtrees below. Concretely, for each tree edge (R_i, R_j) where R_i is the parent and R_j is the child, after all children of R_i have been processed, each tuple $t_i \in R_i^c$ satisfies

$$\mu(t_i) = \prod_{R_j \text{ child of } R_i} \left(\sum_{\substack{t_j \in R_j^c \\ \pi_{K_{ij}}(t_j) = \pi_{K_{ij}}(t_i)}} \mu(t_j) \right),$$

where K_{ij} denotes the shared join attributes on edge (R_i, R_j) . This equation is a recurrence that correct multiplicities must satisfy, not a fresh definition: the semantic meaning of μ was already fixed (the number of global join rows containing t). The recurrence follows

from two observations: (1) the extensions of t_i into different child subtrees combine independently, hence the product over children; and (2) within a single child subtree rooted at R_j , the extensions of t_i are exactly those child tuples $t_j \in R_j^c$ that agree with t_i on the shared key K_{ij} , each already carrying its own count of downstream extensions via $\mu(t_j)$, hence the inner sum. Combined with the leaf base case $\mu(t) = 1$, the recurrence uniquely determines the correct μ at every node.

Circuit Verification. The property the circuit must establish is that the prover-supplied multiplicities satisfy the recurrence stated above: for every internal node R_i and every tuple $t_i \in R_i^c$, $\mu(t_i) = \prod_{R_j \text{ child of } R_i} \sigma_j(t_i)$, where $\sigma_j(t_i) := \sum_{t_j \in R_j^c, \pi_{K_{ij}}(t_j) = \pi_{K_{ij}}(t_i)} \mu(t_j)$ is the aggregated child multiplicity for t_i along edge (R_i, R_j) . The circuit establishes this equality edge by edge, traversing the join tree bottom-up from the leaves so that each parent's equality check is made only after all its children's edges have been processed. For each edge (R_i, R_j) we perform three checks, each enforcing a specific property needed to pin down σ_j :

- (1) **Contiguity of matching children.** Sort R_j^c by K_{ij} using the sort gate (Section 5.2), so child tuples sharing a K_{ij} value become adjacent in the sorted column. This steps turns the set-of-matches condition $\pi_{K_{ij}}(t_j) = \pi_{K_{ij}}(t_i)$ into a contiguous-range condition that downstream gates can scan in a single pass.
- (2) **Correct per-key aggregation.** On the sorted R_j^c , a group-by followed by an aggregation gate (reused from PonglyphDB [26]) produces, for every distinct value v of K_{ij} , the sum $\sigma_j(v) = \sum_{t_j: \pi_{K_{ij}}(t_j) = v} \mu(t_j)$. The output is a key-indexed table $(v, \sigma_j(v))$.
- (3) **Correct parent-side combination.** For each $t_i \in R_i^c$, a lookup argument on K_{ij} retrieves $\sigma_j(t_i[K_{ij}])$ from the table above, and a multiplication gate folds it into a running product maintained per t_i . Once all children of R_i have been processed, a single equality constraint compares this accumulated product against the prover-supplied $\mu(t_i)$.

Correctness follows: if all three checks pass on every edge, then every internal $\mu(t_i)$ equals the product-of-sums value mandated by the recurrence; combined with the leaf base case $\mu(t) = 1$, this forces the prover's annotation to coincide with the unique correct multiplicities.

Final Aggregation. After multiplicities have been propagated to the root relation R_{root}^c , the final aggregate reduces to a single weighted summation over R_{root}^c , but the exact form depends on whether the aggregate attribute lives on the root. We split into two cases.

Case 1: the aggregate attribute lives on the root. When the aggregated attribute A (or the wildcard $\ast$) is an attribute of the root relation, the aggregate is read off from R_{root}^c and its multiplicities via:

- $\text{COUNT}(\ast): \sum_{t \in R_{\text{root}}^c} \mu(t)$
- $\text{SUM}(A): \sum_{t \in R_{\text{root}}^c} \mu(t) \cdot t[A]$
- $\text{AVG}(A): \text{SUM}(A) / \text{COUNT}(\ast)$

Case 2: the aggregate attribute lives on a non-root relation R_i . Here A is not visible at the root, and no formula above applies as stated. We instead compute the partial aggregate $\sum_{t \in R_i^c} \mu(t) \cdot t[A]$ locally at R_i , yielding a single scalar, and propagate only that scalar up to the root along with the usual multiplicity propagation, so no individual A -values ever travel across the tree.

Complexity. For each edge (R_i, R_j) (with R_i the parent and R_j the child), the aggregation requires one sort gate ($O(|R_j^c|)$), one group-by gate ($O(|R_j^c|)$), and one Lookup for multiplicity transfer ($O(|R_i^c| + |R_j^c|)$). Summing over all $k-1$ edges, the total constraint count is $O(IN + OUT)$, where $IN = \sum_i |R_i|$ and OUT is the size of the final aggregated query output.

A worked example demonstrating the full aggregation procedure is provided in Appendix C. And we put the projection and predicates with One-Pass OBJ in Appendix D.

6.4 Output Certification for Plain Joins

R3.O5
C1

Conditions (7)–(10) certify that the clean relations form a coherent joinskeleton, but they do not, by themselves, certify the multiset of output rows reported for a plain (non-aggregate) join. Three cheating strategies remain open to a prover who holds correctly certified clean relations: report a spurious or misassembled row, report a valid row more than once, or omit a valid row. We close all three with a final output-certification step over the certified clean relations $\mathcal{R}^c$ and the reported output, specified in the same style as the previous gates. Private witness. Let $O = (o_1, \dots, o_M)$ denote the reported output rows; M is public, since the output is returned to the verifier. For each row o_j , the prover supplies a auxiliary witness its occurrence vector $v_j = ((idx_{j,1}, t_{j,1}), \dots, (idx_{j,k}, t_{j,k}))$, where $t_{j,i}$ is the clean tuple of R_i^c that contributes to o_j and $idx_{j,i}$ is its row index within R_i^c . Carrying the index (rather than the tuple value alone) is what preserves multiset semantics: if R_i^c containstwo equal tuples, their occurrences are distinguished by index, and each contributes its own copies of the join result.

Verification target. The reported output equals the join of the certified clean relations as a multiset, $O \equiv R_1^c \bowtie \dots \bowtie R_k^c$. Equivalently, the witness must satisfy, for all $j \in [M]$:

$$(idx_{j,i}, t_{j,i}) \in R_i^c, \quad \forall i \in [k], \quad (11)$$

$$t_{j,a}[K_{ab}] = t_{j,b}[K_{ab}], \quad \forall (R_a, R_b) \in T, \quad (12)$$

$$o_j[\text{attr}(R_i)] = t_{j,i}, \quad \forall i \in [k], \quad (13)$$

$$\text{enc}(v_j) < \text{enc}(v_{j+1}), \quad \forall j \in [M-1], \quad (14)$$

$$M = \sum_{t \in R_{\text{root}}^c} \mu(t), \quad (15)$$

where $\text{enc}(v_j) := \sum_{i=1}^k idx_{j,i} \cdot B^{k-i}$ is an order-preserving packing of the index vector for a public base $B > \max_i |R_i|$, and μ is the multiplicity annotation of Section 6.3.

Realization. Each condition is enforced by a primitive already introduced:

- Occurrence Binding (lookup argument) realizes (11). Each clean relation is materialized as an index-augmented table $\{(i, t) : t \text{ is the } i\text{-th tuple of } R_i^c\}$ (compressed to one field element per entry, as in Section 5.2), and the witness column $\{(idx_{j,i}, t_{j,i})\}_j$ is checked against it. This binds every reference to a genuine clean tuple at the claimed index, contributing $O(|R_i^c| + M)$ constraints per relation.
- Join-Key Agreement (equality constraints) realizes (12): one constraint per tree edge per output row, $(k-1) \cdot M$ in total. By the running-intersection property of the jointree, pairwise agreement on all tree edges implies that the referenced tuples assemble into a single consistent tuple over $\mathcal{V}$, i.e., a genuine join result.

- Row Reproduction (equality constraints) realizes (13): the output columns of o_j are wired to the attributes of the referenced tuples, so the reported row is exactly the assembled jointuple, at $O(1)$ constraints per output attribute.
- Canonical Ordering (sort gate) realizes (14). The packing $\text{enc}(v_j)$ is computed by a linear gate; strict increase of consecutive encodings is enforced by range-checking $\text{enc}(v_{j+1}) - \text{enc}(v_j) - 1 \geq 0$, reusing the Sort gate of Section 5.2. Since enc is injective on index vectors, strictly increasing encodings force all occurrence vectors to be pairwise distinct: no join result is reported twice.
- Cardinality Binding (aggregation machinery) realizes (15). The multiplicity propagation of Section 6.3, run on the certified clean instance with leaf multiplicities 1, certifies the join cardinality $\sum_{t \in R_{\text{root}}^c} \mu(t) = |R_1^c| \bowtie \dots \bowtie |R_k^c|$; a single equality constraint compares it against the public M .

Correctness. (11)–(13) imply that every reported row is a genuine element of $R_1^c \bowtie \dots \bowtie R_k^c$ (no spurious or misassembled rows). (14) implies that the M reported occurrence vectors are pairwise distinct, and distinct occurrence vectors generate distinct elements of the join multiset (no duplicates). Hence the reported rows form an M -elements sub-multiset of the join. By (15), M equals the certified join cardinality, so this sub-multiset is the entire join (no omissions). Together with the OBJ guarantee $\mathcal{R}^c = \text{fully reduced instance}$, this pins down $O \equiv J(D)$ exactly.

Complexity. Occurrence Binding costs $O(IN + OUT)$ over all relations; Join-Key Agreement, Row Reproduction, and Canonical Ordering cost $O(OUT)$ for a fixed query; Cardinality Binding reuses the $O(IN + OUT)$ machinery of Section 6.3. The total remains $O(IN + OUT)$.

7 TDJ: Efficient Verification for Cyclic Joins

The previous section developed the OBJ framework for acyclic queries, achieving $O(IN + OUT)$ circuit complexity with full obliviousness. However, many fundamental queries are inherently cyclic, such as cycle detection in graph analytics. Directly applying OBJ does not work, since cyclic queries admit no jointree or union (Section 3.1). In this section, we introduce the **Tree-Decomposed Join** R2.O1

Gate (TDJ), which reformulates the join structure into a tree whose nodes (clusters) absorb the cyclic joins. Once each intra-cluster join is materialized, the inter-cluster structure is acyclic and OBJ applies directly on top of it. The intra-cluster materializations are the only data-dependent piece. In its default mode, TDJ pads each of them to its worst-case intra-cluster bound, so the construction remains fully oblivious—the same guarantee as for acyclic queries—while already reducing the circuit size from $O(IN^k)$ to $O(IN^w)$. As an optional relaxation for settings where full obliviousness is not required, TDJ can instead use a DP-guided upper bound on the intra-cluster circuit capacities. Because the DP-guided capacity sits strictly above the true cardinality while leaking only an (ϵ, δ) -DP amount of information about it, this optional mode relaxes full obliviousness to a quantifiable privacy budget on the released capacities while keeping the join result correct. We defer the formal (ϵ, δ) -obliviousness framework that unifies full obliviousness with this DP-based relaxation, along with the concrete mechanism used to compute the DP capacity bounds, to Appendix G; this section focuses on the TDJ gate itself.

7.1 Tree-Decomposed Join Gate

A natural observation is that many cyclic queries can be re-structured as an acyclic join of materialized sub-joins, in which case OBJ can be applied directly on the outer acyclic structure once each sub-join has been obtained. For example, the triangle query $R_1(A, B) \bowtie R_2(B, C) \bowtie R_3(C, A)$, whose join graph is a 3-cycle, can be re-expressed as a two-way join of the pre-materialized sub-join $R_1 \bowtie R_2$ (a single relation over (A, B, C)) with R_3 on keys (A, C) ; the outer two-relation structure is a tree, so OBJ handles it directly.

Building on this observation, we decompose the query hypergraph into a tree of clusters, where each cluster groups a subset of relations whose intra-cluster join the prover materializes, and the inter-cluster structure is a tree so OBJ applies at that level. The circuit verifies each intra-cluster join using standard binary joining gates from [26]. This decomposition alone already yields a substantial complexity reduction: even with worst-case padding inside each cluster, the overall circuit size drops from $O(IN^k)$ to $O(IN^w)$, where w is the treewidth of the decomposition and is typically far smaller than k . Padding each intra-cluster intermediate to its worst-case bound hides its data-dependent size completely, so this default mode of TDJ preserves full obliviousness; unless a DP mode is explicitly indicated, cyclic-query results reported as “VP Join” in Section 8—in particular, all cyclic VP Join bars in Figure 3—use this fully oblivious mode. Because the intra-cluster materialization sizes are data-dependent, however, worst-case padding can still be expensive. As an optional mode for settings where full obliviousness is not required, TDJ can instead set each intra-cluster capacity to a DP-guided upper bound on the true cardinality, which provides (ϵ, δ) -DP on the released capacity (and hence on the underlying intermediate cardinality) while sitting strictly above the true sizes so that no valid tuple is dropped. We expose this capacity as an abstract parameter in the gate below and defer the noise mechanism and its formal privacy analysis to Appendix G. The gate is expressed in the same style as OBJ:

Definition 3 (Tree-Decomposed Join Gate (TDJ)). Fix a cyclic join query together with a tree decomposition into clusters $\{B_1, \dots, B_p\}$. We define the **Tree-Decomposed Join Gate** through the following three factors:

R2.O1

- Private witness. The input relations, the prover-materialized intra-cluster joins (with each column’s capacity set to its worst-case intra-cluster bound in the default fully oblivious mode, or, optionally, to a DP-guided upper bound on the true cardinality; see Appendix G), padding/dummy indicators for those joins, and the clean/residual partitions used by the inter-cluster OBJ verification.
- Output. A certified clean representation of the cyclic join, equivalently the clean relations or cluster relations from which the final query result is derived. As with OBJ, this output is consumed within the circuit by downstream operators.
- Verification target. The gate certifies that (i) every intra-cluster materialized relation is a correct join of the relations assigned to its cluster, and (ii) the resulting cluster relations satisfy the OBJ-style correctness conditions on the induced acyclic cluster tree.

Realization via cyclic-to-acyclic reduction. To realize this reduction, we adopt the classical technique of tree decomposition [23, 28,

39], which has been widely used in database query processing and worst-case-optimal join algorithms precisely because it confines the combinatorial blowup of a cyclic query to within small local clusters, bounded by the treewidth parameter. Concretely, the query hypergraph is decomposed into a tree of clusters, where each cluster contains a subset of relations whose joins are materialized. Once the cluster tree is obtained, the One-Pass OBJ operates between clusters exactly as in the acyclic case: each cluster acts as a single relation whose tuples are the materialized intra-cluster join results. The inter-cluster Conservation Checks, Membership Checks, Non-Membership Checks, and Pairwise Consistency Checks all carry over directly from Section 6.

Within each cluster, the prover materializes the join of the relations assigned to the cluster before it can participate in the inter-cluster OBJ. For a cluster containing relations $\{R_1, \dots, R_{l_w}\}$, the prover computes the join $R_1 \bowtie \dots \bowtie R_{l_w}$ and provides the result as a witness column. The circuit verifies this intra-cluster join using standard lookup-based binary joining gates [26], which check that each output tuple has matching keys in all participating relations.

If a cluster contains w relations, then its worst-case intermediate size is $O(IN^w)$. Since the treewidth is the minimum width among all tree decompositions of the join hypergraph, a decomposition of width w localizes this blowup to individual clusters rather than the full query. As a result, the worst-case complexity drops from $O(IN^k)$ for the original k -way joint to $O(IN^w)$, where w , the treewidth, is typically much smaller than k . In TDJ’s default, fully oblivious mode, each intra-cluster intermediate is padded to this worst-case bound, which hides the true cardinality completely. Optionally replacing the worst-case bound with a DP-guided bound that is (i) strictly above the true cardinality (so correctness is preserved) and (ii) (ϵ, δ) -DP on the released capacity (so the underlying cardinality remains formally private) is precisely what Appendix G formalizes and instantiates, via the (ϵ, δ) -obliviousness spectrum (Appendix G.1) and the one-sided noise mechanism for cyclic joins (Appendix G.2).

R2.O1

8 Experiments

We evaluate VP Join against PoneglyphDB [26] on both acyclic and cyclic multi-way join queries over TPC-H and graph workloads. Our system is built on Halo2 [49], a state-of-the-art ZK proof system. We compare against PoneglyphDB, the state-of-the-art ZK system for query answering, which uses the same Halo2 backend as VP Join with identical security parameters (Pasta curves and approximately 128-bit security), so any performance gap can be attributed directly to the multi-way join verification strategy. Floating-point values are converted to 64-bit integers before circuit encoding, following prior work [26, 32]. All circuits are implemented in Rust using Halo2 [49].

8.1 Setup

Datasets. We use TPC-H [43] at three scales—60K, 120K, and 240K rows in the lineitem table. For consistency with the PoneglyphDB codebase, on which our implementation builds, only lineitem is scaled while the dimension tables remain at their base sizes; see the discussion of the inherited experimental setup below. The default is 60K. For Q9, following prior work [32], we exclude LIKE predicates, as string operations are orthogonal to join verification. In

R3.O2
R3.Q3

addition, we use three SNAP [31] network datasets. The datasets are LastFM [2], which contains 7,624 nodes and 27,806 edges; Facebook [1], which contains 4,039 nodes and 88,234 edges; and Wikipedia Vote [3], which contains 7,115 nodes and 103,689 edges.

Queries. We select five TPC-H queries (Q3, Q5, Q8, Q9, Q18), spanning 3-to-8-way joins, and four graph queries (GQ1–GQ4) that we design ourselves over the three network datasets. GQ1 and GQ2 are 3- and 4-way path queries, respectively, while GQ3 and GQ4 are 3- and 4-way cycle queries, respectively. For cyclic queries, we further apply tree decomposition before running TDJ. In particular, the tree decompositions used for Q5, GQ3, and GQ4 are described in Appendix H.2.

Hardware and Runtime Environment. All experiments were conducted on a Linux server equipped with dual AMD EPYC 9555 64-core processors (128 cores / 256 threads) and approximately 2 TB of DDR5 memory. For both the evaluated methods, witness generation was executed single-threadedly, while Halo2 proof construction was parallelized across CPU threads. Unless otherwise stated, we used the same thread configuration for both methods. All measured proving times reported in this paper are end-to-end wall-clock measurements under this common runtime setting; numbers labeled as estimates are instead obtained via the extrapolation methodology described below. We omit proof size and verification time, as both are sublinear in circuit size and negligible in practice.

Parameter Setting. PoneglyphDB’s proving times, as reported in [26], can be improved by exploiting primary-key/foreign-key (PK–FK) constraints, that is, prior knowledge of the TPC-H schema, to bound the size of each intermediate join result. However, our design target is a method that works without any schema assumptions beyond the public relation sizes: many real deployments do not expose trustworthy constraints, and workloads such as the graph self-join queries in our experiments have no PK–FK structure at all. As discussed in Section 1, assuming that PK–FK constraints are known to the verifier also corresponds to a simpler setting from the privacy standpoint, since such constraints reveal important structural properties of the joins and may partially expose join selectivities; not relying on them is standard in privacy-preserving query answering [29, 45]. For a fair head-to-head comparison, both systems here therefore use only public-size bounds, so VPJoin’s advantage comes from eliminating intermediates rather than from tighter bounds on them.

For cyclic queries, VPJoin optionally applies DP-guided padding (Appendix G.2), denoted VPJoin+DP. Unless the DP mode is explicitly indicated, every cyclic query result labeled “VPJoin”—in particular, all cyclic VPJoin bars in Figure 3—uses TDJ’s fully oblivious mode with worst-case intra-cluster padding. Unless otherwise stated, the overall privacy budget is $(\epsilon, \delta) = (0.1, 10^{-5})$, split equally across the intra-cluster estimates via basic composition. We run 10 independent rounds and report the average proving time.

Estimation Methodology for PoneglyphDB. Since the fully oblivious PoneglyphDB-style circuits cannot be run to completion even at the default 60K scale, the PoneglyphDB times in Figure 3 are extrapolated from circuit sizes rather than measured on the fully padded executions. Specifically, we track circuit size by the number of gate rows N in our PLONKish/Halo2 backend [19, 49]. Since Halo2

proves over a power-of-two evaluation domain, let $G_C := 2^{\lceil \log_2 N \rceil}$ denote the padded domain size. As discussed in Appendix A, the prover performs both FFT-based polynomial operations and MSMs, and in our implementation the MSM term dominates in practice; proving time therefore scales nearly linearly with the padded domain size, and we use this near-linear scaling as our estimation model. For each query, we follow PoneglyphDB’s binary-join execution strategy, compute the padded intermediate capacities required for full obliviousness, and derive the gate-row count N of the corresponding PLONKish circuit. We anchor each estimate using a measured runnable PoneglyphDB execution on the same server, using the true unpadded intermediate sizes—taken from its released artifact for the TPC-H queries, and from the shared binary-join gates for the graph queries, as described below; this yields N_0 gate rows, padded domain size $G_{C,0} := 2^{\lceil \log_2 N_0 \rceil}$, and wall-clock time T_0 . We then estimate the fully padded proving time as $T_{\text{est}} = T_0 \cdot G_C / G_{C,0}$. This estimate is conservative in PoneglyphDB’s favor: the anchor and the padded circuit share the same gate and column structure, we do not separately charge the $O(N \log N)$ FFT growth, and we omit the extra engineering overheads that arise at very large degrees. The same methodology underlies the motivating estimates quoted in Section 1. The fully oblivious VPJoin times for the cyclic graph queries (GQ3, GQ4), which also exceed feasible runtimes, are estimated by the same circuit-size extrapolation; all other VPJoin times are measured wall-clock times.

We make two further remarks. First, the anchor executions are faster than VPJoin on the same queries: an unpadded PoneglyphDB run materializes only the true intermediate results, which are small for these selective queries, but in doing so it reveals every intermediate cardinality to the verifier. VPJoin’s larger measured times are the price of full obliviousness, and the padded estimates quantify what PoneglyphDB would have to pay for the same guarantee, so Figure 3 compares the two systems at the same privacy level. Second, the released PoneglyphDB artifact provides circuit only for the TPC-H queries. For the graph queries (GQ1–GQ4), we therefore implement the PoneglyphDB-style baseline ourselves as a dedicated binary-join-chain circuit over the same lookup-argument gates (src/graph_sql/pone_baseline.rs in the artifact): every intermediate of the sequential binary-join chain is materialized at a fixed capacity, exactly as in PoneglyphDB’s execution strategy. The anchor is the measured execution of this circuit with capacities set to the true, unpadded intermediate sizes, from which we take the gate-row count and extrapolate to worst-case padding exactly as above.

Inherited Setup from the PoneglyphDB Codebase. Our implementation builds directly on the released PoneglyphDB codebase and, for consistency between the two systems, inherits two simplifications of its benchmarking harness. First, the scaling experiments scale only the line item fact table, while the dimension tables remain at their base sizes rather than being scaled proportionally as TPC-H specifies. Second, the public input of every query proof is trivial (a single constant), and no externally published database commitment $\text{Commit}(D)$ is bound to the proofs: the artifact thus benchmarks the query-verification circuits, with Halo2’s internal IPA commitments providing within-proof binding only (Appendix A). We briefly discuss how the headline numbers would change without

R2.O2
R3.O4
R3.Q2

R2.O1
R3.O6

R3.O1
R3.O3
R3.Q1

R3.O1
R3.O3

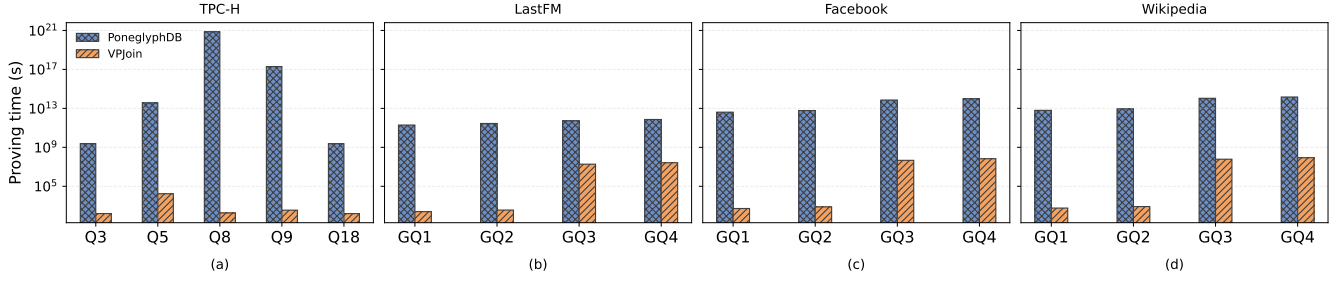


Figure 3: Proving time for 5 TPC-H SQL queries on the TPC-H benchmark and 4 graph SQL queries on 3 network datasets. All PoneglyphDB times are extrapolated estimates obtained via the estimation methodology of Section 8; VPJoin times are measured, except the fully oblivious cyclic graph queries (GQ3, GQ4), which use the same extrapolation.

these simplifications. (i) Scaling the dimension tables proportionally increases the total input size IN ; VPJoin’s proving time grows near-linearly in $IN + OUT$ (Section 8.3), whereas PoneglyphDB’s worst-case-padded intermediates grow multiplicatively with every additional join over the then-larged relations, so dimension-bound joins would become more expensive for both systems in absolute terms, but the reported gap would widen rather than shrink. (ii) Binding Commit(D) to each proof as Appendix A describes adds a one-time commitment cost linear in the database size at setup, amortized across all queries, plus per-proof IPA opening costs; these costs are identical for both systems, since both would open the same committed input columns, and hence do not affect the relative comparison.

8.2 Proving Time

We compare the end-to-end proving time of VPJoin and PoneglyphDB on both TPC-H and graph query workloads. All results are plotted on a logarithmic scale due to the vast difference in magnitude between the two systems.

TPC-H Queries. Figure 3(a) reports the proving time for the five TPC-H queries on the default 60K-row database. VPJoin achieves proving times on the order of 10^3 – 10^4 seconds across all queries. In contrast, PoneglyphDB’s *estimated* proving times range from approximately 10^9 seconds for the 3-way joins (Q3, Q18) to over 10^{20} seconds for Q8, the most complex 8-way join. The gap grows with join arity: for the 3-way joins, VPJoin is roughly 6–7 orders of magnitude faster; for Q5 and Q9 (both 6-way joins), the advantage widens to 9–13 orders of magnitude; and for Q8 (an 8-way join), VPJoin is over 18 orders of magnitude faster.

This dramatic disparity stems directly from the difference in join verification strategy. PoneglyphDB decomposes each multi-way join into a sequence of binary joins and must provision circuit capacity for each materialized intermediate result, padded to worst-case cardinality bounds. For an 8-way join like Q8, the accumulated intermediate blowup renders the circuit large. VPJoin, by contrast, does the verification without materializing any intermediate join result. Among the five queries, the proving time of VPJoin on Q5 stands out because it is the only cyclic TPC-H query (join arity 6) for which VPJoin applies tree decomposition and pads intra-cluster intermediates to worst-case bounds (full obliviousness), resulting in a higher proving time ($\sim 10^4$ s) compared to the acyclic queries ($\sim 10^3$ s). Section 8.4 shows that DP-guided padding (Appendix G.2) reduces this overhead dramatically.

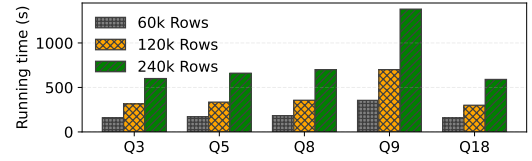


Figure 4: Proving time over data of increasing sizes. Q5 (cyclic) uses DP-guided padding with $\epsilon = 0.1$; all other queries use full obliviousness.

Graph Queries. Figure 3 (b)–(d) report the proving time for GQ1–GQ4 across the three network datasets. For the acyclic path queries (GQ1 and GQ2), VPJoin achieves proving times of approximately 10^3 – $10^{3.5}$ seconds, while PoneglyphDB’s *estimated* times exceed 10^{11} seconds on all three datasets, yielding a gap of 8–9 orders of magnitude. For the cyclic queries (GQ3 and GQ4), VPJoin with full obliviousness (worst-case padding) reaches approximately 10^7 seconds (*estimated*), while PoneglyphDB’s *estimated* times reach 10^{12} – $10^{13.5}$ seconds, giving VPJoin a clear 4–6 order-of-magnitude advantage.

Across all three datasets, the relative performance pattern remains consistent: the gap is widest for acyclic queries (GQ1, GQ2), where VPJoin derives the greatest benefit from its compact join-aware construction, and narrows for cyclic queries (GQ3, GQ4), where worst-case padding for intra-cluster intermediates results in increased circuit capacity. As the dataset size grows from LastFM to Wikipedia, PoneglyphDB’s *estimated* times increase more rapidly than VPJoin’s, reflecting the multiplicative overhead of intermediate result materialization on larger inputs.

8.3 Scalability

We evaluate how VPJoin’s proving times scale with increasing database size. Figure 4 reports the proving time for all five TPC-H queries at three scales: 60K, 120K, and 240K rows in the line item table; as described in Section 8, only line item is scaled, with the dimension tables fixed at their base sizes. Since Q5 is the only cyclic TPC-H query, its fully oblivious cost is dominated by worst-case intra-cluster padding (see Section 8.4); to make the scalability trend comparable, we report Q5 with DP-guided padding at $\epsilon = 0.1$.

Across all queries, doubling the input size approximately doubles the proving time. For example, Q3 increases from roughly 155 s (60K) to 315 s (120K) to 600 s (240K), and Q18 follows a similar trajectory from 155 s to 300 s to 590 s. This near-linear scaling is

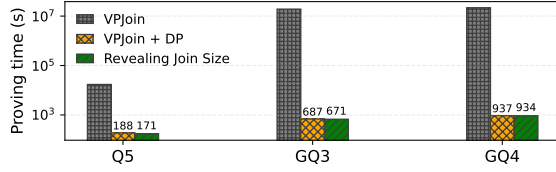


Figure 5: Impact of privacy constraints on proving time. The fully oblivious VPJoin times for GQ3 and GQ4 are extrapolated estimates (Section 8). (The legend of this figure is corrected in this revision: the previous version mistakenly swapped the labels of the fully oblivious VPJoin bars and the Revealing Join Size bars.)

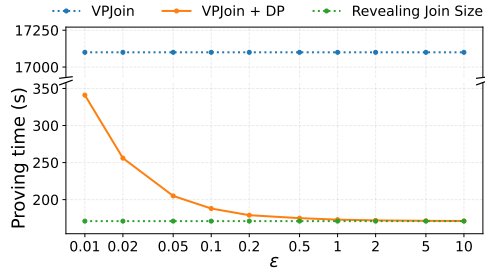


Figure 6: Proving time of Q5 for various ϵ values.

consistent with the theoretical OBJ complexity of $O(IN + OUT)$ for acyclic queries.

Q5 is at approximately 170 s/335 s/660 s, and Q8 at approximately 180 s/355 s/700 s. Notably, Q8 is an 8-way join yet its proving time is comparable to Q3 (a 3-way join). This demonstrates a key advantage of our verification approach: the proving cost is decoupled from the number of joins, scaling only with the data volume.

8.4 Privacy–Efficiency Trade-off for Cyclic Joins

For acyclic queries, VPJoin achieves full obliviousness at $O(IN + OUT)$ cost. Cyclic queries, however, require tree decomposition, which introduces intra-cluster intermediates. Padding to these worst-case bounds preserves full obliviousness but incurs substantial proving overhead. We therefore reevaluate the DP-guided padding strategy (Appendix G.2), which replaces worst-case bounds with DP-noisy cardinality estimates, and compare three privacy regimes:

- VPJoin: circuit capacities are set to worst-case upper bounds, revealing no information about intermediate cardinalities.
- VPJoin + DP: circuit capacities are set using noisy cardinality bounds satisfying (ϵ, δ) -DP, with controlled leakage.
- Revealing Join Size: circuit capacities match the true intermediate cardinalities, providing no privacy protection on join sizes but serving as a lower bound on proving time.

R2.O3 Overall comparison of the three regimes. Figure 5 reports the proving time of the three cyclic queries (Q5, GQ3, GQ4) under each regime. Full obliviousness (VPJoin) incurs proving times of approximately 17,100 s for Q5 and exceeding 10^7 s (estimated) for GQ3 and GQ4, reflecting the cost of worst-case padding on decomposed intermediates. In contrast, VPJoin + DP ($\epsilon = 0.1$) reduces the proving time to 188 s (Q5), 687 s (GQ3), and 937 s (GQ4)—improvements of roughly 2, 4, and 4 orders of magnitude, respectively. These DP-guided times are reclosely to the Revealing Join Size baseline (171 s, 671 s,

934 s), incurring at most 10% overhead. This demonstrates that DP-guided padding nearly eliminates the cost of obliviousness for cyclic joins while still providing formal (ϵ, δ) -DP guarantees on the leaked circuit dimensions.

Various privacy budgets. Figure 6 shows VPJoin + DP proving time as a function of ϵ for Q5, with the fully oblivious and Revealing Join Size baselines as reference lines. The DP curve decreases smoothly from approximately 340 s at $\epsilon = 0.01$ to 171 s at $\epsilon \geq 2$, converging to the no-privacy baseline. At the default $\epsilon = 0.1$, the proving time is approximately 188 s, only about 10% above the lower bound. The graph queries GQ3 and GQ4 exhibit the same trend: the DP-guided proving time decreases monotonically as ϵ grows and converges to the Revealing Join Size baseline once ϵ reaches 1–2, at which point the added noise becomes negligible. Per-dataset breakdowns are provided in Appendix I.

References

- [1] 2024. SNAP: Facebook Ego-Network Dataset. <https://snap.stanford.edu/data/ego-Facebook.html>.
- [2] 2024. SNAP: LastFM Social Network Dataset. <https://snap.stanford.edu/data/feather-lastfm-social.html>.
- [3] 2024. SNAP: Wikipedia Vote Network Dataset. <https://snap.stanford.edu/data/wiki-Vote.html>.
- [4] Mahmoud Abo Khamis, Hung Q Ngo, and Atri Rudra. 2016. FAQ: Questions asked frequently. In Proceedings of the 35th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems. 13–28.
- [5] Arvind Arasu, Ken Eguro, Raghav Kaushik, Donald Kossmann, Pingfan Meng, Vineet Pandey, and Ravi Ramamurthy. 2017. Concerto: A high concurrency key-value store with integrity. In Proceedings of the 2017 ACM International Conference on Management of Data. 251–266.
- [6] Guillaume Bagan, Arnaud Durand, and Etienne Grandjean. 2007. On acyclic conjunctive queries and constant delay enumeration. In Computer Science Logic: 21st International Workshop, CSL 2007. Springer, 208–222.
- [7] Sumeet Bajaj and Radu Sion. 2013. CorrectDB: SQL engine with practical query authentication. Proceedings of the VLDB Endowment 6, 7 (2013), 529–540.
- [8] Stephanie Bayer and Jens Groth. 2012. Efficient zero-knowledge argument for correctness of a shuffle. In EUROCRYPT. 263–280.
- [9] Catriel Beeri, Ronald Fagin, David Maier, and Mihalis Yannakakis. 1983. On the desirability of acyclic database schemes. J. ACM 30, 3 (1983), 479–513.
- [10] Jonathan Bootle, Andrea Cerulli, Pyrros Chaidos, Jens Groth, and Christophe Petit. 2016. Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting. In Advances in Cryptology—EUROCRYPT 2016: 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8–12, 2016, Proceedings, Part II 35. Springer, 327–357.
- [11] Sean Bowe, Jack Grigg, and Daira Hopwood. 2019. Recursive proof composition without a trusted setup. Cryptology ePrint Archive (2019).
- [12] Sean Bowe, Jack Grigg, and Daira Hopwood. 2020. Recursive proof composition without a trusted setup. Cryptology ePrint Archive, Report 2019/1021. <https://eprint.iacr.org/2019/1021>.
- [13] Sean Bowe, Ying Tong Lai, Daira Hopwood, and Jack Grigg. [n.d.]. pasta_curves: implementation of the Pallas and Vesta (Pasta) curve cycle. https://github.com/zcash/pasta_curves. Accessed: 2026-04-18.
- [14] Wei Dong, Zijun Chen, Qiyao Luo, Elaine Shi, and Ke Yi. 2024. Continual observation of joins under differential privacy. Proceedings of the ACM on Management of Data 2, 3 (2024), 1–27.
- [15] Cynthia Dwork. 2006. Differential Privacy. In Automata, Languages and Programming, 33rd International Colloquium, ICALP 2006, Venice, Italy, July 10–14, 2006, Proceedings, Part II (Lecture Notes in Computer Science), Michele Bugliesi, Bart Preneel, Vladimiro Sassone, and Ingo Wegener (Eds.). Springer, 1–12. https://doi.org/10.1007/11787006_1
- [16] Cynthia Dwork. 2006. Differential privacy. In International colloquium on automata, languages, and programming. Springer, 1–12.
- [17] C. Christopher Erway, Alptekin Küpçü, Charalampos Papamanthou, and Roberto Tamassia. 2009. Dynamic Provable Data Possession. In Proceedings of the 16th ACM Conference on Computer and Communications Security (CCS). 213–222.
- [18] Ariel Gabizon and Zachary J Williamson. 2020. plookup: A simplified polynomial protocol for lookup tables. Cryptology ePrint Archive (2020).
- [19] Ariel Gabizon, Zachary J Williamson, and Oana Ciobotaru. 2019. Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive (2019).
- [20] Craig Gentry and Daniel Wichs. 2011. Separating Succinct Non-Interactive Arguments from All Falsifiable Assumptions. In Proceedings of the 43rd Annual ACM Symposium on Theory of Computing (STOC). 99–108.
- [21] Shafi Goldwasser, Yael Tauman Kalai, and Guy N Rothblum. 2015. Delegating computation: interactive proofs for muggles. Journal of the ACM (JACM) 62, 4 (2015), 1–64.
- [22] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. 1985. The Knowledge Complexity of Interactive Proof-Systems (Extended Abstract). In Proceedings of the 17th Annual ACM Symposium on Theory of Computing, May 6–8, 1985, Providence, Rhode Island, USA, Robert Sedgewick (Ed.). ACM, 291–304. <https://doi.org/10.1145/22145.22178>
- [23] Georg Gottlob, Nicola Leone, and Francesco Scarcello. 1999. Hypertree decompositions and tractable queries. In Proceedings of the eighteenth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems. 21–32.
- [24] Jens Groth. 2016. On the size of pairing-based non-interactive arguments. In Annual international conference on the theory and applications of cryptographic techniques. Springer, 305–326.
- [25] Binbin Gu. 2025. Integrating Zero-Knowledge Proofs for Efficient and Verifiable Data Systems. Ph.D. Dissertation. University of California, Irvine.
- [26] Binbin Gu, Juncheng Fang, and Faisal Nawab. 2025. PoneglyphDB: Efficient Non-interactive Zero-Knowledge Proofs for Arbitrary SQL-Query Verification. Proceedings of the ACM on Management of Data 3, 1 (2025), 1–27.
- [27] Ulrich Haböck. 2022. Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive (2022).
- [28] Manas R Joglekar, Rohan Puttagunta, and Christopher Ré. 2016. Ajar: Aggregations and joins over annotated relations. In Proceedings of the 35th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems. 91–106.
- [29] Noah Johnson, Joseph P Near, and Dawn Song. 2018. Towards practical differential privacy for SQL queries. Proceedings of the VLDB Endowment 11, 5 (2018), 526–539.
- [30] Joe Kilian. 1992. A Note on Efficient Zero-Knowledge Proofs and Arguments (Extended Abstract). In Proceedings of the 24th Annual ACM Symposium on Theory of Computing (STOC). 723–732.
- [31] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. <https://snap.stanford.edu/data>.
- [32] Xiling Li, Chenkai Weng, Yongxin Xu, Xiao Wang, and Jennie Rogers. 2023. ZKSQL: Verifiable and Efficient Query Evaluation with Zero-Knowledge Proofs. Proceedings of the VLDB Endowment 16, 8 (2023), 1804–1816.
- [33] Silvio Micali. 2000. Computationally Sound Proofs. SIAM J. Comput. 30, 4 (2000), 1253–1298.
- [34] Payman Mohassel and Saeed Sadeghian. 2013. How to hide circuits in MPC an efficient framework for private function evaluation. In Annual International Conference on the Theory and Applications of Cryptographic Techniques. Springer, 557–574.
- [35] C Andrew Neff. 2001. A verifiable secret shuffle and its application to e-voting. In CCS. 116–125.
- [36] Charalampos Papamanthou, Roberto Tamassia, and Nikos Triandopoulos. 2008. Authenticated Hash Tables. In Proceedings of the 15th ACM Conference on Computer and Communications Security (CCS). 437–448.
- [37] Bryan Parno, Jon Howell, Craig Gentry, and Mariana Raykova. 2016. Pinocchio: Nearly practical verifiable computation. Commun. ACM 59, 2 (2016), 103–112.
- [38] Benny Pinkas, Thomas Schneider, Oleksandr Tkachenko, and Avishay Yanai. 2019. Efficient circuit-based PSI with linear communication. In Annual International Conference on the Theory and Applications of Cryptographic Techniques. Springer, 122–153.
- [39] Neil Robertson and Paul D Seymour. 1986. Graph minors. II. Algorithmic aspects of tree-width. Journal of Algorithms 7, 3 (1986), 309–322.
- [40] Rohit Sinha and Mihai Christodorescu. 2018. Veritasdb: High throughput key-value store with integrity. Cryptology ePrint Archive (2018).
- [41] Roberto Tamassia. 2003. Authenticated data structures. In Algorithms-ESA 2003: 11th Annual European Symposium, Budapest, Hungary, September 16–19, 2003. Proceedings 11. Springer, 2–5.
- [42] Justin Thaler. 2013. Time-Optimal Interactive Proofs for Circuit Evaluation. In Advances in Cryptology - CRYPTO 2013. 71–89.
- [43] Transaction Processing Council. 2023. TPC-H Benchmark. <http://www.tpc.org/tpch/>. Accessed: 2023-xx-xx.
- [44] Chenghong Wang, Lina Qiu, Joes Bater, and Yukui Luo. 2024. SPECIAL: Synopsis Assisted Secure Collaborative Analytics. arXiv preprint arXiv:2404.18388 (2024).
- [45] Yilei Wang and Ke Yi. 2021. Secure yannakakis: Join-aggregate queries over private data. In Proceedings of the 2021 International Conference on Management of Data. 1969–1981.
- [46] Yin Yang, Dimitris Papadias, Stavros Papadopoulos, and Panos Kalnis. 2009. Authenticated join processing in outsourced databases. In Proceedings of the 2009 ACM SIGMOD International Conference on Management of data. 5–18.

- [47] Mihalis Yannakakis. 1981. Algorithms for acyclic database schemes. In *VLDB*, Vol. 81. 82–94.
- [48] Andrew Chi-Chih Yao. 1986. How to generate and exchange secrets. In *27th annual symposium on foundations of computer science (Sfcs 1986)*. IEEE, 162–167.
- [49] Zcash. [n.d.]. halo2. <https://github.com/zcash/halo2>
- [50] Zcash. [n.d.]. Halo2 Protocol. <https://zcash.github.io/halo2/design/protocol.html>
- [51] Ce Zhang, Cheng Xu, Haixin Wang, Jianliang Xu, and Byron Choi. 2021. Authenticated keyword search in scalable hybrid-storage blockchains. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 996–1007.
- [52] Yupeng Zhang, Daniel Genkin, Jonathan Katz, Dimitrios Papadopoulos, and Charalampos Papamanthou. 2017. vSQL: Verifying arbitrary SQL queries over dynamic outsourced databases. In *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 863–880.
- [53] Yupeng Zhang, Daniel Genkin, Jonathan Katz, Dimitrios Papadopoulos, and Charalampos Papamanthou. 2017. A zero-knowledge version of vSQL. *Cryptology ePrint Archive* (2017).
- [54] Yupeng Zhang, Jonathan Katz, and Charalampos Papamanthou. 2015. IntegriDB: Verifiable SQL for outsourced databases. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*. 1480–1491.
- [55] Qingji Zheng, Shouhuai Xu, and Giuseppe Ateniese. 2012. Efficient query integrity for outsourced dynamic databases. In *Proceedings of the 2012 ACM Workshop on Cloud computing security workshop*. 71–82.

	(private) columns			public column	
	A	B	I	O	polynomial constraints:
row 1:	1	12	0	30	row 1: $I_1 - 0 = 0$
row 2:	2	20	12		row 2: $B_1 + I_1 - I_2 = 0$
row 3:	3	8	22		row 3: $B_2 + I_2 - I_3 = 0$
row 4:			30		row 4: $B_3 + I_3 - I_4 = 0$

→ denotes a copy constraint. For each row in the matrix, the multivariate polynomials are evaluated to 0.

Figure 7: PLONKish circuit illustration for SQL query “SELECT SUM(B) FROM R”.

A Proof Generation and Verification

We instantiate the proof system using the Halo2 framework [49, 50], the same backend used by PoneglyphDB [26], and inherit its cryptographic guarantees. We choose Halo2 for three reasons. First, its PLONKisharithmetization [19] supports custom gates that let us encode complex join-verification logic into compact, application-specific constraints. Second, it provides native lookup arguments [18], which our design exploits heavily for range checks and set-membership proofs. Third, PoneglyphDB [26] has already designed efficient gate primitives for relational operations (range checks, sorting, etc.) on the same backend, which we directly reuse and extend.

Setup. Halo2 operates over the Pasta curve cycle [11, 13] (255-bit field, $\lambda = 128$ -bit security) and uses the Inner Product Argument (IPA) [12] for polynomial commitments, providing a transparent setup with no trusted third party.

Proof Complexity. In this paper, we use circuit size to refer to the total number of enforced constraints (equivalently, the number of circuit rows up to constant factors in our Halo2 implementation). We analyze each gate by its constraint cost and obtain the total circuit size by summing the costs of all instantiated gates. Let n denote the size of a circuit. The Halo2 IPA-based proof system achieves the following asymptotic bounds:

- Proof size: $O(\log n)$ group elements, yielding proofs of a few tens of kilobytes in practice.
- Verification time: $O(\log n)$ group operations, completing in under one second regardless of query complexity.
- Proving time: $O(n \log n)$. The prover performs $O(n \log n)$ field operations for polynomial arithmetic (FFT/NTT) and $O(n)$ elliptic-curve group operations for multi-scalar multiplications (MSMs) used in polynomial commitments and the IPA opening proof [12, 19]. Since group operations are orders of magnitude more expensive than field operations, the $O(n)$ MSM term dominates in practice [26], and the observed proving time scales almost linearly with n .

The sublinear proof size and verification time make the system succinct: the verifier’s cost is independent of the computation being proved. Since proving time is dominated by MSM and thus scales

nearly linearly with the circuit size, minimizing the number of constraints is the primary optimization objective, and more importantly, the central motivation for our witness-guided approach that avoids materializing intermediate join results.

Database commitment via IPA. The commitment $\text{Commit}(D)$ published during Setup (Section 4) is realized using the same IPA-based polynomial commitment scheme employed by the proof system itself, so no extra cryptographic primitive or assumption is introduced. Concretely, the prover canonicalizes the database $D = \{R_1, \dots, R_k\}$ into a single coefficient vector $\mathbf{d} = (d_0, \dots, d_{N-1}) \in \mathbb{F}^N$ over the Pasta scalar field (in the canonical layout, relations are ordered by name and each attribute value becomes one coordinate of $\mathbf{d}$, which keeps the encoding injective; compressing each multi-attribute tuple to one field element, matching the lookup convention of Section 5.2, is an optional space optimization), and computes the Pedersen-style vector commitment

$$\text{Commit}(D) = \sum_{i=0}^{N-1} d_i G_i + rH,$$

where $\{G_i\}, H$ are independent Pasta-curve generators fixed by the transparent setup [13, 49] and $r \in \mathbb{F}$ is a uniformly random blinding scalar. This is a single multi-scalar multiplication of $N+1$ terms, so the prover’s commitment work is $O(N)$ while the output is a single group element ($O(1)$ space). Binding follows from the discrete-logarithm hardness of the Pasta curve, and hiding follows from the uniformity of r [10, 12].

Commitment verification. At query time, every subsequent ZK-SNARK proof is generated with respect to $\text{Commit}(D)$: the in-circuit witness values for the input relations are tied back to $\mathbf{d}$ via standard IPA opening proofs [10, 12, 19], which certify claimed evaluations of $f_D(X) = \sum_i d_i X^i$ at verifier-chosen challenge points. Each opening proof has size $O(\log N)$ group elements; checking a single opening performs an $O(N)$ multi-scalar multiplication via the standard IPA reduction [12], and the $O(\log N)$ verification cost is available in the amortized setting through Halo-style accumulation [11]; we reuse the protocol verbatim and inherit its security analysis. Because $\text{Commit}(D)$ is fixed before any query is issued, the binding property ensures every accepted proof is computed over the same database: to switch to any $D' \neq D$ that still opens to $\text{Commit}(D)$, the prover would have to produce a second pre-image of a Pedersen commitment, which is infeasible under discrete-log hardness. This is what discharges Threat (i) of the threat model (Section 4): any attempt by the prover to switch to a different database silently is caught at verification.

Artifact status. The originally released artifact benchmarks the query-verification circuits with a trivial public input and does not bind the externally published $\text{Commit}(D)$ to each query proof (see the inherited-setup discussion in Section 8). Halo2’s internal IPA polynomial commitments provide within-proof binding automatically, but not the binding to a stable, externally published commitment described in this appendix. The updated artifact implements this commitment layer as a standalone, additive module (`src/column_commit.rs`), leaving every query circuit unchanged so that its cost is measured separately and added on top of any query’s

cost. Rather than committing the whole database as one long vector, it commits each attribute column separately over the same evaluation domain the query circuits already use ($k = 16$ for our TPC-H circuits, which accommodate the 60K-row line item): the commitment layer then reuses the circuits' existing parameters verbatim, needs no separate large setup, and yields one published commitment per column, matching the circuits' witness columns one-to-one. Each column is committed with a fresh random blinder, $C_j = \text{Commit}(f_j; r_j)$ with $f_j(X) = \sum_i \text{col}_j[i] X^i$, so the published commitments are hiding.

Binding a query proof to these commitments is then a random-point consistency check, applied per query to exactly the columns that query's circuit witnesses (8–19 columns for our TPC-H queries; the two Edge columns for the graph queries), in that circuit's own degree ($k = 16$ for TPC-H, $k = 17$ –18 for the graph queries). A public challenge x is derived by Fiat–Shamir from the published commitments, the layout, and the query proof; the prover opens each committed column x via an IP opening proof revealing $y_j = f_j(x)$, while the circuit evaluates its own witness column x by a Horner accumulation ($O(\text{rows})$ constraints per column) and exposes y_j as a public output. Since two distinct columns agree at a random x only with probability $\deg/\mathbb{F}$, matching values certify that the circuit's witness column equals the committed data, which is precisely the in-circuit consistency step that discharges Threat (i). Measured per query on the experiment server at the 60K scale: publishing a query's column commitments takes at most 0.07 s once at setup (at most 608 bytes, for Q8's 19 columns), and per proof the layer adds 1.0–2.2 s of in-circuit prover time over the identical circuit without binding, plus 2.0–4.7 s for the column openings and at most 0.25 s of verification—at a total of 3.0–6.8 s across Q3–Q18 and 1.5–2.9 s across GQ1–GQ4 on all three graph datasets, i.e., under 4% of the corresponding measured VPJoin proving times. The per-query measurements, including the baseline-versus-bound deltas, are emitted as a CSV by `src/bin/query_commit_bench.rs`.

B PLONKish Circuit Example

Example B.1. Figure 7 illustrates a PLONKish circuit that evaluates the SQL query `SELECT SUM(B) FROM R`. The relation R consists of two attributes, A and B . The circuit uses two advice columns, A and B , to store the input tuples.

To perform the summation, we introduce an additional advice column I that stores intermediate partial sums. We denote the value in column $X \in \{A, B, I, O\}$ at row i (1-indexed) by X_i .

In row 1, we initialize the accumulator by setting $I_1 = 0$ and enforce the polynomial constraint $I_1 - 0 = 0$, which ensures that the summation starts from zero. For each subsequent row $i \geq 2$, the circuit adds the previous value in column B to the running sum. Specifically, in row 2 we add B_1 to I_1 and store the result in I_2 . This is enforced by the constraint $B_1 + I_1 - I_2 = 0$. Similarly, in rows 3 and 4, the circuit enforces $B_2 + I_2 - I_3 = 0$ and $B_3 + I_3 - I_4 = 0$, respectively, thereby accumulating the sum of all values in column B .

Finally, the value I_4 , which equals $\sum_{i=1}^3 B_i$, is copied into the public instance column at position O_1 using a copy (equality) constraint. This allows the verifier to read the final output of the computation while all intermediate values remain private.

C Aggregation Example

Example C.1 (Aggregation via tuple multiplicities). Continuing from Example 6.2 with the same clean relations $R_1^c = \{(1,1), (2,1)\}$, $R_2^c = \{(1,11)\}$, $R_3^c = \{(11,4), (11,2)\}$ and joint tree rooted at R_2 , we compute `SELECT A, COUNT(*), SUM(D) ... GROUP BY A` without materializing the join result (see Figure 8).

- (1) Initialize leaf multiplicities. Every tuple in a leaf relation gets $\mu = 1$: $R_1^c: (1,1) \mapsto 1, (2,1) \mapsto 1$; $R_2^c: (11,4) \mapsto 1, (11,2) \mapsto 1$.
- (2) Process edge (R_2, R_1) on B . Sort R_1^c by B and group: key $B = 1$ has $\Sigma\mu = 1 + 1 = 2$. Lookup into R_2^c : tuple $(1,11)$ receives $\mu_L = 2$.
- (3) Process edge (R_2, R_3) on C . Sort R_3^c by C and group: key $C = 11$ has $\Sigma\mu = 1 + 1 = 2$. In the same pass, compute the partial aggregate $\Sigma\mu \cdot D = 1 \cdot 4 + 1 \cdot 2 = 6$ for key $C = 11$. Lookup into R_2^c : tuple $(1,11)$ receives $\mu_R = 2$ and partial $\text{SUM}(D) = 6$.
- (4) Multiply at root. $\mu(1,11) = \mu_L \times \mu_R = 2 \times 2 = 4$.
- (5) Group-by and final aggregates. Since the query groups by A and $A \in R_1$ (a leaf), per-group aggregates are computed at R_1^c . Each tuple $(a,1) \in R_1^c$ matches the root tuple $(1,11)$, whose right-branch multiplicity is $\mu_R = 2$ and partial $\text{SUM}(D) = 4 + 2 = 6$.
 - $A = 1$: $\text{COUNT}(\ast) = 1 \cdot \mu_R = 2$, $\text{SUM}(D) = 1 \cdot 6 = 6$.
 - $A = 2$: $\text{COUNT}(\ast) = 1 \cdot \mu_R = 2$, $\text{SUM}(D) = 1 \cdot 6 = 6$.

AVG(D) per group is simply $\text{SUM}(D) / \text{COUNT}(\ast) = 6 / 2 = 3$. These match the full join result $\{(1,1,11,4), (2,1,11,4), (1,1,11,2), (2,1,11,2)\}$ grouped by A : each group has 2 tuples with $D \in \{4,2\}$, giving $\text{COUNT} = 2$ and $\text{SUM}(D) = 6$ —all computed without materializing the join.

D Projection and Predicates with One-Pass OBJ

Projection. Many analytical queries project the join result onto a subset of attributes (e.g., `SELECT R1.A, R2.B FROM R1 JOIN R2 JOIN R3 ...`). In existing ZK database systems, projection is applied only after the full join is materialized, so the circuit must still accommodate the entire $O(IN^k)$ intermediate. In our framework, the OBJ produces clean relations $\mathcal{R}^c$ that are subsets of the original input tables. When the query is a free-connex acyclic query—that is, the output attributes form a connected subset within the jointree—the final join result can be enumerated directly from the clean relations, and projection is applied during this enumeration by copying only the required attribute columns to the output; **thereported (projected) output is certified against the clean relations by the output-certification step of Section 6.4**. Relations outside the output subtree contribute only to the semi-join reductions (filtering non-participating tuples) but their non-key attributes never appear in the output circuit. Since projection merely selects a subset of columns from the clean relations and the enumerated output, it introduces no additional constraints beyond those already required by the One-Pass OBJ and the final join enumeration.

In-Relation Predicates. Queries frequently include selection predicates on a single relation—for example, `WHERE R1.price > 100`. Because each such predicate references only attributes within one input table, it can be evaluated as a per-tuple filter before the OBJ begins: the prover incorporates the predicate into the witness split, placing tuples that fail the condition directly into the residual set. The circuit verifies each tuple in R_i against the predicate with a single arithmetic constraint (e.g., `range-check gate` for inequalities), adding at most $O(|R_i|)$ constraints per relation—fully absorbed into the existing $O(IN)$ bound.

R3.05

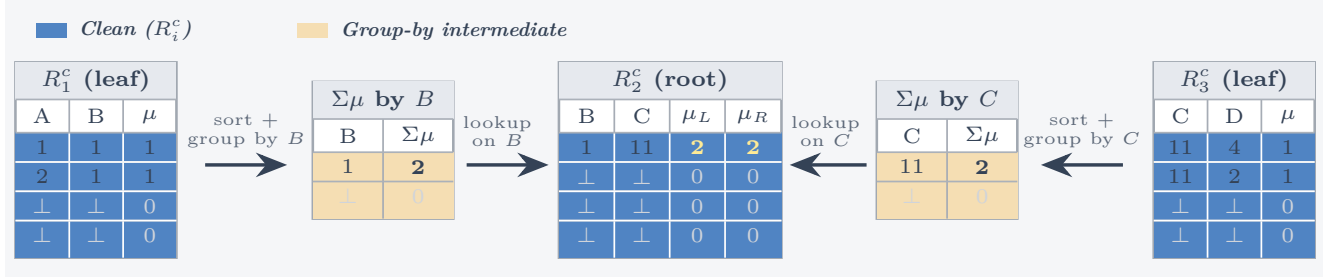


Figure 8: Aggregation via tuple multiplicities for $\text{SELECT } A, \text{COUNT}(*), \text{SUM}(D) \text{ FROM } R_1, R_2, R_3 \text{ WHERE } R_1.B = R_2.B \text{ AND } R_2.C = R_3.C \text{ GROUP BY } A$.

R3.O5 Cross-Relation Predicates. Queries often include selection predicates that reference attributes from multiple relations—for example, $\text{WHERE } R_1.\text{price} > R_2.\text{budget}$. In existing systems, such predicates can only be evaluated after materializing the binary join that brings these attributes together, again requiring $O(IN^k)$ worst-case circuit capacity. In contrast, in our framework, cross-relation predicates are enforced within the output-certification step of Section 6.4, which assembles each reported output row from its clean tuples. For each output tuple—which combines attributes from multiple clean relations—the circuit enforces the predicate as an additional arithmetic constraint (e.g., a range check for inequality predicates or an equality gate for exact matches). Since these constraints are applied per output row, they add at most $O(OUT)$ constraints and do not affect the overall $O(IN + OUT)$ complexity. Importantly, no intermediate relation needs to be materialized to evaluate these predicates: the clean relations already identify which input tuples participate, and the predicate is checked only on the final combined output.

E Correctness and Obliviousness Analysis

E.1 Two-Pass OBJ

Correctness. To prove that $\{R_1^c, \dots, R_k^c\}$ contains exactly the tuples participating in the global join, we verify two properties. Note that these are properties of the verified join result, distinct from the proof-system-level soundness and completeness defined in Section 3.2.

(1) **No false positives** (every retained tuple is valid). For each semi-join gate on edge (P, C) , the Membership Check enforces $\pi_{K_{PC}}(R_P^c) \subseteq \pi_{K_{PC}}(R_C)$, so only tuples with matching keys survive. By induction on the join tree, the bottom-up and top-down passes implement the Yannakakis semi-join reduction, and the pairwise consistency check confirms key agreement on every edge. Thus every tuple in R_i^c is fully connected in the global join.

(2) **No false negatives** (no valid tuple is discarded). For each semi-join gate on edge (P, C) , the Conservation Check ensures $R_P \equiv R_P^c \cup R_P''$ (not tuples lost), and the Non-Membership Check enforces $\pi_{K_{PC}}(R_P'') \cap \pi_{K_{PC}}(R_C) = \emptyset$. If a matching tuple were placed in R_P'' , it would violate non-membership, a contradiction. Since each gate preserves all valid tuples and the two-pass reduction is exhaustive for acyclic queries, the final $\{R_1^c, \dots, R_k^c\}$ contain every tuple participating in the global join.

Obliviousness. The OBJ gate achieves obliviousness through data-independent circuit structuring; each relation occupies a column

of its own public size $|R_i|$, with no uniform worst-case padding across relations.

- **Structural Uniformity.** The number of advice columns, their lengths, and the polynomial constraints are determined solely by the query, the public schema, and the public relation sizes $\{|R_1|, \dots, |R_k|\}$. The wiring of the permutation and lookup arguments is fixed at compile time and does not branch on tuple values.
- **Data Hiding via Private Witness.** Tuple values are hidden by the zero-knowledge property of the underlying commitments scheme (IPA commitments in Halo2). The prover supplies the per-edge partition (R_i', R_i'') as private witnesses, so the verifier sees only commitments to columns of public length $|R_i|$ without learning their contents.
- **Cardinality Hiding.** The splitsizes $|R_i'|$ and $|R_i''|$ are data-dependent, but together they occupy exactly $|R_i|$ rows, a public quantity. Their individual values appear only as positions within these $|R_i|$ -row columns and are masked by the private witness, so the verifier learns nothing about join selectivity or intermediate join cardinalities.

E.2 One-Pass OBJ

Correctness. To prove that $\mathcal{R}^c$ is the correctly reduced state, i.e., that the clean relations contain exactly those tuples participating in the global join, we verify two properties (as with the Two-Pass variant, these concern the verified join result, not the proof system itself).

(1) **No false positives** (every clean tuple is valid). (9) enforces that for every edge (R_i, R_j) in the join tree, $\pi_{K_{ij}}(R_i^c) = \pi_{K_{ij}}(R_j^c)$. By induction on the tree structure, this implies that every tuple in any R_i^c has a matching join partner in every adjacent clean relation. Thus, every tuple in the provided clean set participates in the global join.

(2) **No false negatives** (no valid tuple is hidden). We show by contradiction that no valid joining tuple is hidden in $\mathcal{R}^c$. Suppose there exists a tuple $t \in R_i^c$ (for some relation i) that participates in a valid global join result r . Consider any edge (R_i, R_j) incident to R_i , and let $t_v = t[K_{ij}]$ be the join key value on that edge. By (8), $t \notin R_i^c$, and by (9) applied in reverse, if $t_v \in \pi_{K_{ij}}(R_j^c)$ then $t_v \in \pi_{K_{ij}}(R_i^c)$; so the R_j -component of r must also lie in R_j^c . Propagating along the tree, every component of r must reside in the residual set of its relation, yielding a complete join tuple within $\mathcal{R}^c$. But this contradicts (10), which enforces $R_k^c \times (\dots (R_2^c \times R_1^c) \dots) = \emptyset$.

Therefore, $\mathcal{R}^c$ contains all contributing tuples. Combined with (7) ($R_i \equiv R_i^c \cup R_i''$), this proves that $\mathcal{R}^c$ is the correct result.

Obliviousness. The One-Pass OBJ maintains obliviousness under the same principles:

- For each relation R_i , the prover supplies R_i^c and R_i^r as private witnesses. Since the partition is exhaustive ($|R_i^c| + |R_i^r| = |R_i|$), the two parts together occupy exactly $|R_i|$ rows, which is public. The actual split $|R_i^c|/|R_i^r|$ is hidden within these rows and inaccessible to the verifier through the zero-knowledge commitments.
- The four invariants (7)–(10) are enforced by Permutation, Membership, and Non-Membership checks whose circuit layouts depend only on the public sizes $\{|R_1|, \dots, |R_k|\}$. The verifier's computational path is constant and independent of the clean/residual ratio, so the query selectivity is never leaked through the circuit.

F Detailed Complexity Analysis

This appendix gives the full constraint-cost derivation. Instead of padding every relation to a uniform capacity, we keep each relation at its actual public size $|R_i|$ and express every cost as a function of these sizes. All totals are then expressed in terms of the total input size $IN = \sum_{i=1}^k |R_i|$.

F.1 Primitive Costs

Let $B = 64$ denote the key bit width and $b = 8$ the range-check chunk size, giving $t = \lceil B/b \rceil = 8$ range-check chunks per value. Since every column length is at most IN , each primitive check runs in $O(IN)$ constraints; we spell out the per-column expression to expose the constant factors used later.

- ConservationCheck(permutation argument verifying multiset equality of two columns of the same length n): n constraints, hence $O(IN)$.
- MembershipCheck with witness column X and table column Y (Plookup): $\max(|X|, |Y|)$ constraints, hence $O(IN)$.
- Non-MembershipCheck on columns X and Y via merge-and-sort has two parts:
 - a permutation argument binding the prover-supplied sorted sequence to the merge of X and Y : $|X| + |Y|$ constraints;
 - a sortedness proof range-checking consecutive differences: $t(|X| + |Y|) = 8(|X| + |Y|)$ constraints.

Standalone, a Non-Membership Check thus costs $9(|X| + |Y|) = O(IN)$. When the gate already contains a Conservation Check (a permutation argument on the same rows), the Non-Membership permutation merges with it under PLONK-ish's global permutation argument, so only the $8(|X| + |Y|) = O(IN)$ sortedness cost is added on top.

F.2 Semijoin Gate Cost

A semijoin gate $SJ_{i \rightarrow j}$ verifying the partition of R_i against R_j invokes:

- (1) Conservation on R_i vs. $R_i^r \cup R_i'' : |R_i|$ constraints.
- (2) Membership $\pi_{K_{ij}}(R_i^c) \subseteq \pi_{K_{ij}}(R_j)$: $\max(|R_i^c|, |R_j|) \leq \max(|R_i|, |R_j|)$ constraints.
- (3) Non-Membership $\pi_{K_{ij}}(R_i'') \cap \pi_{K_{ij}}(R_j) = \emptyset$: because the gate already contains a Conservation permutation, only the sortedness cost is added: $8(|R_i''| + |R_j|) \leq 8(|R_i| + |R_j|)$ constraints.

Summing:

$$\mathcal{C}(SJ_{i \rightarrow j}) \leq |R_i| + \max(|R_i|, |R_j|) + 8(|R_i| + |R_j|) = O(|R_i| + |R_j|). \quad (16)$$

F.3 Two-Pass OBJ Total

The Two-Pass OBJ applies one semijoin gate per edge in each of the bottom-up and top-down passes, for $2(k-1)$ gates total. Summing (16) over all edges:

$$\mathcal{C}_{2\text{-Pass}} \leq 2 \sum_{(P,C) \in T} O(|R_P| + |R_C|) = O\left(\sum_{v \in T} \deg(v) |R_v|\right),$$

where $\deg(v)$ is the degree of node v in the joint tree. Since $\sum_v \deg(v) = 2(k-1)$, we have $\sum_v \deg(v) |R_v| \leq d_{\max} IN$ where $d_{\max} = \max_v \deg(v)$. For a fixed query (constant k , constant tree shape):

$$\mathcal{C}_{2\text{-Pass}} = O(IN).$$

F.4 One-Pass OBJ Total

The One-Pass OBJ enforces four invariants over the prover-supplied partition $(\mathcal{R}^c, \mathcal{R}^r)$ with $|R_i^c| + |R_i^r| = |R_i|$.

(1) Conservation (k checks). One Conservation Check per relation on column length $|R_i|$:

$$\sum_{i=1}^k |R_i| = IN.$$

(2) Non-Membership (k checks). A standalone Non-Membership Check on R_i^c and R_i^r (no sibling Conservation on the same rows) incurs both the permutation and sortedness parts. The merged sequence has length $|R_i^c| + |R_i^r| = |R_i|$, costing $9|R_i|$ per check. Summing over all relations:

$$\sum_{i=1}^k 9|R_i| = 9IN.$$

(3) Pairwise Consistency ($2(k-1)$ checks, optional). For every edge (R_i, R_j) , two Membership Checks verify set equality on the shared key. Each costs $\max(|R_i^c|, |R_j^c|) \leq \max(|R_i|, |R_j|)$. Total:

$$O\left(\sum_{(i,j) \in T} (|R_i| + |R_j|)\right) = O(IN).$$

This step may be omitted for aggregation-only workloads (Section 6.3).

(4) Residual Disconnectedness ($k-1$ semijoin gates). A bottom-up semijoin reduction on $\mathcal{R}^r$ applies one semijoin gate per edge. By (16), each costs $O(|R_i^r| + |R_j^r|) \leq O(|R_i| + |R_j|)$. Total:

$$O\left(\sum_{(i,j) \in T} (|R_i| + |R_j|)\right) = O(IN).$$

Total. Summing the four components (Conservation, Non-Membership, optional Pairwise Consistency, and Residual Disconnectedness):

$$\mathcal{C}_{1\text{-Pass}} = IN + 9IN + O(IN) + O(IN) = O(IN).$$

F.5 Comparison

Both variants achieve $O(IN)$ circuit complexity, so the advantage of the One-Pass OBJ lies in the constant factor. The key difference is how the top-down pass is replaced:

- Two-Pass: each top-down edge invokes a full semi-join gate on the reduced parent and child, at cost $O(|R_P| + |R_C|)$ per edge (dominated by the Non-Membership sortedness over $|R_P'| + |R_C|$).
- One-Pass: the top-down pass is replaced by k standalone Non-Membership Checks, one per relation. Because R_i^c and R_i^r partition R_i , each merge-and-sort is over $|R_i|$ elements (not $|R_P'| + |R_C|$), costing $9|R_i|$. Summed over all relations: $9IN$.

Under uniform relation sizes $|R_i| = N$ with $IN = kN$, each semi-join gate costs $N + N + 8 \cdot 2N = 18N$, and each standalone Non-Membership Check costs $9N$. The totals reduce to

$$\mathcal{C}_{2\text{-Pass}} = 36(k-1)N, \quad \mathcal{C}_{1\text{-Pass}} = 18(k-1)N + 9kN = (27k-18)N.$$

The One-Pass saves $9(k-2)N$ constraints for $k \geq 3$, and in addition eliminates intermediate column allocations from the top-down pass entirely: the circuit operates directly on the prover-supplied partition.

G Differential Privacy Framework for Cyclic Joins

R2.01 Section 7.1 introduced the **Tree-Decomposed Join Gate (TDJ)** and, as an optional relaxation of its default fully oblivious mode, allowed each intra-cluster witness column to replace worst-case padding with a DP-guided upper bound on the true cardinality. This appendix provides the formal underpinning for that construction. Appendix G.1 formalizes the connection between obliviousness and differential privacy, unifying full obliviousness and its DP-based relaxation on a single (ϵ, δ) -obliviousness spectrum. Appendix G.2 then instantiates this spectrum for cyclic joins, specifying the concrete one-sided noise mechanism **TDJ** uses and analyzing its privacy and correctness. **Appendix G.3 discusses why extending PoneglyphDB with the same DP relaxation would remain substantially more expensive than VPJoin + DP.**

G.1 Obliviousness and Differential Privacy

Continuing from the previous subsection, worst-case intra-cluster padding is still far from satisfactory: for queries with non-trivial treewidth, the $O(IN^w)$ padding can remain prohibitively expensive. This cost is not incidental to our realization but an inherent consequence of requiring full obliviousness: since the witness capacity allocated to each intermediate is part of the public circuit (see Section 3.2), it must be fixed independently of the database, otherwise a curious verifier could read off the true intermediate cardinalities directly from the published circuit. OBJoin cyclic queries satisfy this requirement essentially for free, since the circuit avoids materializing intermediate joins altogether. For cyclic queries, however, the intra-cluster intermediates introduced by tree decomposition have data-dependent cardinalities, so preserving full obliviousness forces each intra-cluster witness capacity to be padded to its worst case, $O(IN^w)$.

To achieve a more favorable intermediate size, and hence more efficient proving, a principled relaxation of full obliviousness is necessary. DP is a natural candidate precisely because it shares the same structural form as full obliviousness: both are constraints on how much a mechanism's output varies when the input database changes by a single record. Full obliviousness demands that the output be identical for any two neighboring databases, whereas DP relaxes

“identical” to “probabilistically indistinguishable within a quantifiable (ϵ, δ) budget.” This shared form lets us place the two notions on a single spectrum of structural leakage, which we formalize next.

Definition 4 (Differential Privacy). A randomized mechanism $\mathcal{M}$ satisfies (ϵ, δ) -differential privacy [16] iff for any two neighboring databases D_1, D_2 that differ by a single record and any set of outcomes S ,

$$\Pr[\mathcal{M}(D_1) \in S] \leq e^\epsilon \cdot \Pr[\mathcal{M}(D_2) \in S] + \delta.$$

The privacy budgets (ϵ, δ) control the level of the privacy guarantee, where smaller values correspond to stronger privacy protection.

Embedding full obliviousness into a DP framework. Obliviousness itself is a deterministic notion: it is defined purely by requiring identical circuits across same-parameter databases. However, we can embed it into a DP framework so that full obliviousness and a DP-based relaxation sit on a single spectrum. View the circuit generator $\mathcal{G}(\cdot)$ as a mechanism from databases to circuits. It can be shown that any $\mathcal{G}(\cdot)$ satisfying full obliviousness is $(0, 0)$ -DP: since $\mathcal{G}(\cdot)$ produces the same output on any two databases sharing the same public parameters, we have $\Pr[\mathcal{G}(D_1) \in S] = \Pr[\mathcal{G}(D_2) \in S]$ for every neighboring pair D_1, D_2 and every set S . As a result, we propose to unify full obliviousness and its DP-based relaxation under a single notion, which we call (ϵ, δ) -obliviousness: a circuit generator $\mathcal{G}(\cdot)$ is (ϵ, δ) -oblivious whenever it satisfies (ϵ, δ) -DP as a mechanism from databases to circuits. This yields a unified spectrum of structural leakage parameterized by (ϵ, δ) :

- $\epsilon = 0, \delta = 0$ (deterministic): full obliviousness, where circuit dimensions are fully data-independent (with worst-case padding).
- $0 < \epsilon < \infty, \delta$ negligible: DP-guided relaxation, where circuit capacities satisfy (ϵ, δ) -DP.
- $\epsilon \rightarrow \infty$: no privacy, where capacities equal the true cardinalities.

We emphasize that (ϵ, δ) parameterize the relaxation, not obliviousness itself: full obliviousness remains a deterministic equality-based notion. Within this framework, ZK protects the content of witness columns, while DP relaxes full obliviousness on the observable circuit dimensions and still ensures formal privacy. We instantiate the middle regimen next.

G.2 DP-Guided Relaxation for Cyclic Joins

With the DP-guided relaxation of full obliviousness (Appendix G.1), we are now able to set each intra-cluster circuit capacity to a data-dependent upper bound on the true cardinality, rather than its worst-case bound $O(IN^w)$. The capacity must upper-bound the true size, since underestimating would discard valid tuples and break correctness. To guarantee this upper-bound property while still providing formal privacy, we adopt the one-sided noise mechanism of Wang et al. [44], which yields (ϵ, δ) -DP on the released capacity and ensures privacy of the intermediate cardinalities.

Incentive-aligned privacy model. A key observation is that in our ZK database setting, the DP noise protects the prover's own data from the verifier. This creates a natural alignment of incentives: any manipulation of the DP mechanism—using too little noise, too much noise, or incorrect sensitivity estimates—harms only the prover, either through privacy loss or increased computation. The verifier is unaffected, since it only cares about the correctness of the

final query result, not the intermediate sizes. This incentive alignment allows us to omit in-circuit verification of DP parameters entirely: the circuit need not prove that noise was sampled correctly or that it meets any threshold. These are the prover's concern alone.

DP upper bound realization. To obtain the DP upper bound on the cardinality of the in-cluster join result m , one natural idea is to adopt the one-sided noise mechanism [44], which guarantees that the introduced DP noise is non-negative. The noise scale can be computed from the maximum frequency of each relation, following the analysis in [14]. To preserve privacy during the computation of the noisy scale, these maximum frequencies must themselves be made DP-compliant, again by applying the one-sided noise mechanism. We denote the resulting DP upper bound on m by $\tilde{m}$. By construction, $\tilde{m} \geq m$ while preserving (ϵ, δ) -DP.

Circuit realization. The circuit implementation is straightforward. The prover sets each intra-cluster witness column capacity to the noisy bound. The circuit processes all rows uniformly, using standard “is_dummy” selector columns to exclude padding from downstream computations. No additional constraints are needed for the DP mechanism itself. We emphasize that the DP mechanism is a prover-side step that is not verified in-circuit: the circuit consumes only the final padded capacity and never samples noise itself. The capacities are computed offline by a prover-side routine that, for each intra-cluster join, calibrates the noise scale via the join mechanism of [14] (which adapts to the join via truncation) and applies it as a one-sided, non-negative upper bound, so the released $\tilde{m} \geq m$ never underestimates the true cardinality. The originally released artifact hard-coded the resulting capacities rather than shipping this offline routine, which is why they appeared there as fixed constants; the updated artifact includes the generation routine (dp/noise_generator.py, with a Rust twin src/dp_noise.rs callable at witness-generation time, so DP-mode circuit stake here released capacity directly as their intra-cluster column size). The routine first releases one-sided noisy truncation thresholds $\tilde{F}_A, \tilde{F}_B$ for the per-relation maximum join-key frequencies (each of which has global sensitivity 1), then releases the capacity with one-sided noise calibrated to the join-size sensitivity $\max(\tilde{F}_A, \tilde{F}_B)$: under the standard neighboring definition, changing one tuple of A moves the join size by at most $\tilde{F}_B$ and changing one tuple of B by at most $\tilde{F}_A$, so the sensitivity is $\max(\tilde{F}_A, \tilde{F}_B)$, with the released upper bounds in place of the private frequencies. This holds for the self-join clusters (the Edge $\bowtie$ Edge bags of the cyclic graph queries) as well, treating each aliased instance as its own relation. Since the threshold upper-bound the true maximum frequencies with certainty, the truncated join coincides with the true join, the calibration is data-independent given the released thresholds, the total cost follows by basic composition, and the released capacity never underestimates the true cardinality.

Correctness. The correctness of TDJ follows from two independent arguments. First, the inter-cluster join is verified by the One-Pass OBJ (Section 6.2.2), whose correctness is established in Section 6. Second, each intra-cluster join is verified using binary join gates [26] that enforce key matching via lookup arguments. The DP

mechanism affects only the capacity (padding size) of the intra-cluster witness columns, not the correctness constraints themselves: the one-sided noise guarantees $\tilde{m} \geq m$, so all valid tuples fit within the allocated capacity and no results are lost.

G.3 Comparison with a Hypothetical PoneglyphDB + DP Extension

Since DP-guided padding relaxes full obliviousness, a natural question is whether the baseline could benefit equally: one could imagine extending PoneglyphDB with the same relaxation, replacing each worst-case-padded intermediate of its binary-join chain with a DP upper bound on the intermediate's cardinality. Such a hypothetical PoneglyphDB + DP extension would remain substantially slower than VPJoin + DP for cyclic joins, for two reasons rooted in how the two systems structure the join. R2.O1

First, the noise magnitude scales with the join sensitivity. PoneglyphDB must decompose the whole k -way join into $k-1$ binary joins and release a DP-protected capacity after each of them. Because a single input tuple can join with tuples in every downstream relation, the sensitivity of a deep intermediate grows exponentially with the number of joins below it, so PoneglyphDB's noise must be calibrated to the sensitivity of (up to) the full k -way join. VPJoin, in contrast, confines materialization to the small clusters of the decomposition tree: inside a cluster it uses the same binary-join gates as PoneglyphDB, but the inter-cluster joins are verified by OBJ without materializing any intermediate. Consequently, VPJoin only adds noise proportional to the sensitivity of a partial join, i.e., the join inside a single cluster, which is far smaller.

Second, the privacy budget must stretch much further in PoneglyphDB + DP. Because PoneglyphDB forms a chain of binary joins, a tuple in one relation is involved in all $k-1$ of them, so the overall (ϵ, δ) budget must be divided across all $k-1$ releases. A larger sensitivity or a smaller per-step privacy budget directly increases the DP noise added to each intermediate capacity; since the circuit must be padded to this noisy upper bound, more noise means more dummy rows, a larger circuit, and therefore a longer proving time. VPJoin materializes only a small constant number of clusters, so its budget is split only a few ways. Together, these two effects imply that the gap between the two systems grows with the join depth even when both are granted the same DP relaxation.

H Queries

The complete 4 graph queries are shown below:

H.1 Graph Queries

GQ1: 3-way path.

```
SELECT COUNT(*) AS cnt
FROM Edger1
  JOIN Edger2 ON r1.dst = r2.src
  JOIN Edger3 ON r2.dst = r3.src
WHERE r1.src < r2.src AND r2.src < r3.src;
```

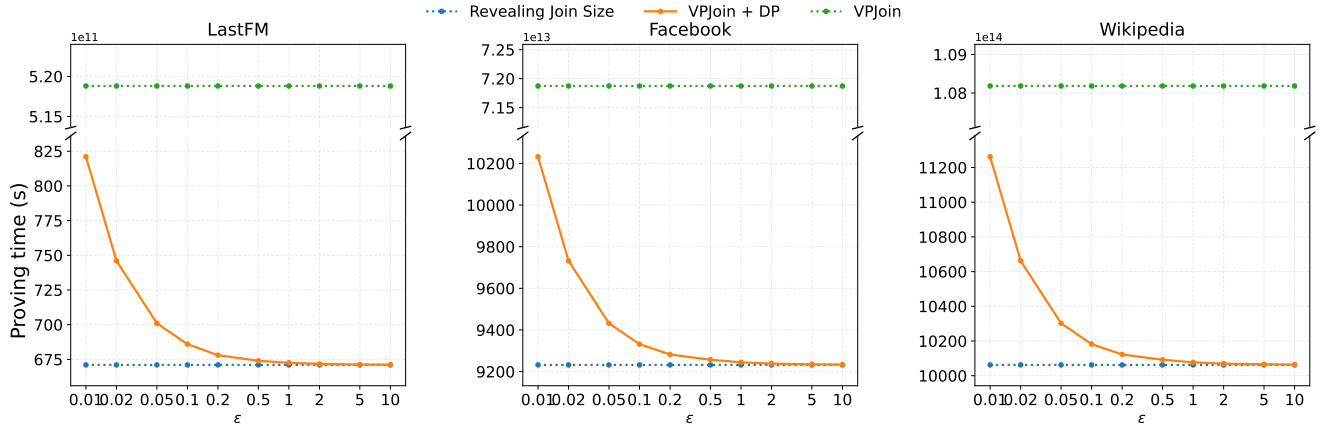


Figure9:ProvingtimeofgraphqueryGQ3forvariousepsilonvalueson3networkdatasets.

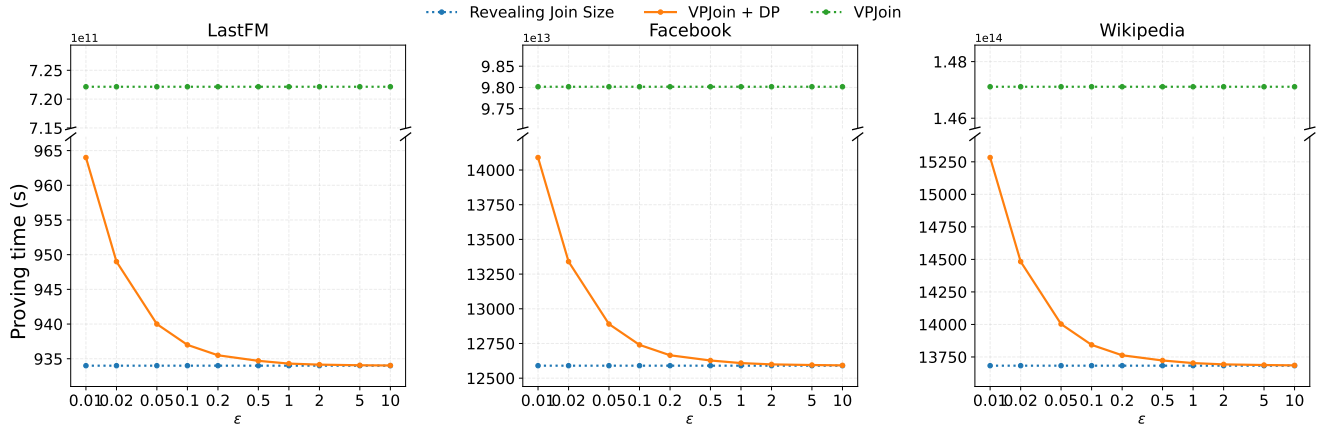


Figure10:ProvingtimeofgraphqueryGQ4forvariousepsilonvalueson3networkdatasets.

GQ2:4-waypath.

```

SELECTCOUNT(*)AScnt
FROMEdger1
JOINEdger2ONr1.dst=r2.src
JOINEdger3ONr2.dst=r3.src
JOINEdger4ONr3.dst=r4.src
WHEREr1.src<r2.src
ANDr2.src<r3.src
ANDr3.src<r4.src;

```

GQ3:3-waycycle.

```

SELECTCOUNT(*)AScnt
FROMEdger1
JOINEdger2ONr1.dst=r2.src
JOINEdger3ONr2.dst=r3.src
ANDr3.dst=r1.src
WHEREr1.src<r2.srcANDr2.src<r3.src;

```

GQ4:4-waycycle.

```

SELECTCOUNT(*)AScnt
FROMEdger1
JOINEdger2ONr1.dst=r2.src
JOINEdger3ONr2.dst=r3.src
JOINEdger4ONr3.dst=r4.src
ANDr4.dst=r1.src
WHEREr1.src<r2.src
ANDr2.src<r3.src
ANDr3.src<r4.src;

```

H.2 Cyclic Query Decomposition

We describe how the three cyclic queries (Q5, GQ3, GQ4) are decomposed into acyclic cluster trees for the **TDJ** gate (Section 7.1).

TPC-HQ5. We abbreviate the six tables as *C* (customer), *O* (orders), *L* (lineitem), *S* (supplier), *N* (nation), *R* (region), and write *ok*, *ck*, *sk*, *nk*, *rk* for orderkey, custkey, suppkey, nationkey, regionkey. Q5 is a 6-way join whose joining graph contains a 4-way cycle among *C*,

O, L , and S : they are connected through both the transactional path ($O.ok = C.ck, L.ok = O.ok, L.sk = S.sk$) and the shared nationality constraint ($C.nk = S.nk$). The remaining tables N and R join acyclically via nk and rk and thus do not require materialization into bags. We break the cycle by grouping the four cyclic relations into two bags:

- Bag 1 $\{O, C\}$: materialize $O \bowtie C$ with the date filter, producing tuples (ok, nk) .
- Bag 2 $\{L, S\}$: materialize $L \bowtie S$, producing tuples $(ok, nk, price, disc)$.

R2.O1 The inter-cluster joint tree has four nodes—Bag 1, Bag 2, N , R : Bag 2 joins Bag 1 on (ok, nk) , Bag 2 joins N on nk , and N joins R on rk . This tree is acyclic, so the One-Pass OBJ verifies all inter-cluster joins. Only the two bags require intra-cluster materialization; **their padding capacities are set to their worst-case intra-cluster bounds in TDJ's default fully oblivious mode, or, optionally, to DP-guided noisy bounds (Appendix G.2).**

GQ3 (3-way cycle / triangle). GQ3 counts triangles $A \rightarrow B \rightarrow C \rightarrow A$ with $A < B < C$ over the Edge(src, dst) relation, using three self-join instances r_1, r_2, r_3 . The cycle arises from the closing edge $r_3.dst = r_1.src$. We decompose via the separator (A, C) :

- Bag 1 $\{r_1, r_2\}$: materialize the 2-hop path $A \rightarrow B \rightarrow C$ (wedges).
- Bag 2 $\{r_3\}$: the closing edge $C \rightarrow A$.

These separator variables are (A, C) . Bag 2 sends a message to Bag 1: for each distinct (A, C) key, the count of closing edges $C \rightarrow A$. The final answer aggregates over Bag 1, weighting each wedge by the Bag 2 count and enforcing $A < B < C$.

GQ4 (4-way cycle). GQ4 counts 4-cycles $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$ with $A < B < C < D$ over the Edge(src, dst) relation, using four self-join instances r_1, r_2, r_3, r_4 . The cycle arises from $r_4.dst = r_1.src$. We decompose via a diagonal cut on (A, C) :

- Bag 1 $\{r_1, r_2\}$: materialize the path $A \rightarrow B \rightarrow C$.
- Bag 2 $\{r_3, r_4\}$: materialize the path $C \rightarrow D \rightarrow A$.

These separator variables are (A, C) . Bag 2 sends a message to Bag 1: for each (A, C) key, the count of paths $C \rightarrow D \rightarrow A$ satisfying $C < D$. The final answer aggregates over Bag 1, weighting each path $A \rightarrow B \rightarrow C$ by the Bag 2 count and enforcing $A < B < C$.

I Detailed

Privacy Budget Sensitivity for Graph Queries

This section provides a detailed, per-dataset analysis of the privacy budget sensitivity for the cyclic graph queries GQ3 and GQ4, complementing the summary presented in Section 8.4. For each query, Figures 9 and 10 plot the proving time of three configurations—Revealing Join Size (blue, dashed), VPJoin + DP (orange, solid), and VPJoin with full obliviousness (green, dashed)—as a function of ϵ on the LastFM, Facebook, and Wikipedia Voted datasets. Note that the y-axes use a broken scale: the upper portion shows the (estimated) VPJoin proving time and the lower portion shows the VPJoin + DP and Revealing Join Size proving times.

GQ3 (3-way cycle). Figure 9 reports the ϵ -sensitivity for GQ3. On LastFM (27,806 edges), the VPJoin + DP proving time decreases from approximately 820 s at $\epsilon = 0.01$ to 670 s at $\epsilon \geq 2$, converging to the Revealing Join Size baseline of roughly 670 s. The fully oblivious VPJoin time is estimated at approximately 5.18×10^{11} s,

which is entirely infeasible. On Facebook (88,234 edges), the DP curve ranges from approximately 10,200 s ($\epsilon = 0.01$) down to 9,200 s ($\epsilon \geq 2$), against a Revealing Join Size baseline of roughly 9,200 s. The VPJoin estimate of 7.19×10^{13} s. On Wikipedia (103,689 edges), the DP curve ranges from approximately 11,200 s ($\epsilon = 0.01$) to 10,100 s ($\epsilon \geq 2$), with the baseline at roughly 10,050 s and the VPJoin estimate at 1.08×10^{14} s.

Across all three datasets, the DP overhead at the default $\epsilon = 0.1$ is modest: roughly 4% above the Revealing Join Size baseline on LastFM, 2% on Facebook, and 3% on Wikipedia. This overhead arises from the additional padding grown into introduced by the Laplace noise, which translated directly into extra circuit constraints. As ϵ increases beyond 1, the noise magnitude becomes negligible relative to the true intermediate cardinalities, and the DP curve converges to the no-privacy baseline.

GQ4 (4-way cycle). Figure 10 reports the analogous results for GQ4. On LastFM, the DP curve decreases from approximately 964 s ($\epsilon = 0.01$) to 934 s ($\epsilon \geq 2$), converging to the Revealing Join Size baseline of roughly 934 s. The VPJoin estimate is approximately 7.21×10^{11} s. On Facebook, the range is approximately 14,200 s ($\epsilon = 0.01$) to 12,600 s ($\epsilon \geq 1$), with a baseline of roughly 12,600 s and a VPJoin estimate of 9.80×10^{13} s. On Wikipedia, the DP curve ranges from approximately 15,250 s ($\epsilon = 0.01$) to 13,750 s ($\epsilon \geq 2$), against a baseline of roughly 13,750 s and a VPJoin estimate of 1.47×10^{14} s.

Compared to GQ3, the absolute proving times for GQ4 are higher across all datasets, reflecting the additional cost of decomposing and padding a 4-way cycle versus a 3-way cycle. However, the relative DP overhead remains small: at $\epsilon = 0.1$, the overhead is approximately 1% on LastFM, 7% on Facebook, and 5% on Wikipedia. The convergence behavior is consistent with GQ3: moderate privacy budgets ($\epsilon \geq 0.5$) bring the DP-guided proving time within a few percent of the no-privacy lower bound, while the fully oblivious alternative remains infeasible by over 8 orders of magnitude.